

Dokumentasjon for Fagprøve (Bekkelaget IL)

Skrevet av: Kristian Haugsrud

Dokumentasjon for Fagprøve (Bekkelaget IL).....	1
Oppgaven.....	4
1. Introduksjon.....	4
2. Oppgaven.....	5
3. Krav til løsningen.....	5
4. Opsjoner.....	6
5. Arbeidsform.....	6
Planlegging.....	6
Min tolkning av oppgaven.....	6
Hvorfor klubben trenger dette akkurat nå?.....	7
2. Krav til applikasjonen.....	8
Den åpne publikumsdelen.....	8
Det lukkede redaktør panelet.....	8
2.1. Brukerhistorier.....	9
For vanlige besøkende og foreldre:.....	9
For redaktører og administratorer:.....	9
3. Generelt.....	9
4. Generelle tanker om oppgaven.....	9
Techstack.....	10
Frontend.....	10
Backend og Database.....	11
Vurdering av arkitektur og serverlogikk.....	11
Drøfting av autentiseringsmetode.....	12
Verktøy/applikasjoner.....	12
Autentisering.....	12
Styring og opprettelse av redaktører.....	13
Implementasjon av sikkerhet.....	13
Sikkerhet og personvern.....	13
Database struktur.....	14
Tabelloversikt.....	14
Entity Relationship Diagram (ERD).....	14
Relasjoner.....	15
Arbeidsmetodikk.....	16
Design og layout.....	16
Dag til dag plan.....	18
Tirsdag 26. Mai – Dag 1.....	18
Onsdag 27. Mai – Dag 2.....	18
Torsdag 28. Mai – Dag 3.....	19
Fredag 29. Mai – Dag 4.....	19
Mandag 1. Juni – Dag 5.....	19

Tirsdag 2. Juni – Dag 6.....	19
Onsdag 3. Juni – Dag 7.....	19
Gjennomføring.....	19
Tirsdag 26. Mai – Dag 1.....	19
Onsdag 27. Mai – Dag 2.....	22
Torsdag 28. Mai – Dag 3.....	25
Fredag 29. Mai – Dag 4.....	32
Mandag 1. Juni – Dag 5.....	38
Tirsdag 2. Juni – Dag 6.....	43
Onsdag 3. Juni – Dag 7.....	46
Egenvurdering.....	46
Hva jeg har lært.....	46
Hva jeg kunne gjort annerledes.....	47
Planleggingen.....	47
Gjennomføringen.....	47
Plan for videre utvikling.....	47
GDPR og personvern.....	47
E-posthåndtering.....	48
Eksport av medlemsregisteret.....	48
Maks antall plasser og venteliste.....	48
Bildeopplasting og mediebehandling.....	48
Endringslogg for redaktører.....	48
Admin-dashboard med statistikk.....	48
Læremål.....	48
Planlegge, utvikle og dokumentere løsninger med innebygd personvern og sikkerhet.	48
Utvikle og bruke dokumentasjon og veiledninger.....	49
Velge og bruke relevante rammeverk og moduler til utvikling.....	49
Feilsøke kode og rette feil i algoritmer og kode.....	49
Utvikle og tilpasse brukergrensesnitt som ivaretar krav til universell utforming.....	49
Håndtere påloggingsopplysninger på en sikker og forsvarlig måte.....	49

Oppgaven

1. Introduksjon

Bekkelaget IL er en breddeklubb i Ottestad med flere idretter under samme tak, blant annet fotball, hockey, håndball, innebandy og sykling. Klubben har et aktivt miljø og en stab av frivillige som har holdt det gående med en Facebook side som primær kanal i flere år. Det fungerer dårligere og dårligere. Innleggene drukner i feeden, kommende arrangementer er nesten umulig å finne, og styret holder fortsatt orden på medlemmer og betalt kontingent i en Excel fil som flyter rundt på e post.

Klubben har nå tatt en avgjørelse om å skaffe seg en ordentlig nettside. De vil ha et sted der medlemmer og foreldre kan finne nyheter, terminlister og arrangementer, og der styret kan håndtere medlemsregisteret og publisere innhold uten å være avhengig av at noen henger over Facebook hele dagen.

2. Oppgaven

Du skal bygge en webapplikasjon for Bekkelaget IL. Den skal ha to deler. En åpen del der besøkende får informasjon om klubben og kan melde seg på arrangementer, og en innlogget del der redaktører fra klubben publiserer innhold og holder orden på medlemsregisteret.

Du har stor frihet i hvordan du løser dette, både teknisk og hvordan du modellerer dataene. Det viktigste er at klubben faktisk får dekket sine behov. Tenk gjennom hvem som vil bruke nettsiden i praksis. Det er foreldre som vil sjekke når neste kamp er, det er en kasserer som vil markere innbetalt kontingent uten å åpne en regnearkkolonne, og det er noen i styret som vil legge inn et nytt arrangement etter en lang dag på jobb. Brukervennlighet i begge ender teller.

3. Krav til løsningen

Den åpne delen skal:

- Ha en forside som gjør det tydelig hva klubben er og hva som skjer.
- Vise nyheter og artikler fra klubben.
- Vise en oversikt over kommende arrangementer.
- La besøkende melde seg på et arrangement uten å være innlogget.

Den innloggede delen skal:

- Ha innlogging for redaktører i klubben.
- La en redaktør opprette, endre og slette artikler og arrangementer.
- La en redaktør se hvem som har meldt seg på et arrangement.
- Inneholde et medlemsregister der admins kan legge til og endre medlemmer.

- La en redaktør markere medlemskontingent som betalt eller ubetalt for et medlem. Betalingen håndteres utenfor systemet, det er kun status som registreres her.
- Admins kan legge til redaktører og admins

I tillegg gjelder følgende generelle krav:

- Hele kodebasen skrives i TypeScript.
- Du velger selv hvilken stack som passer best, og begrunner valget.
- Kildekoden ligger i Git og er tilgjengelig gjennom hele prøveperioden.
- Applikasjonen er deployet og åpent tilgjengelig fra internett.
- Løsningen er responsiv og fungerer godt både på mobil og desktop.
- Koden er ryddig og vedlikeholdbar, med dokumentasjon der det er hensiktsmessig.

Dokumentasjonen skal følge arbeidet underveis, ikke skrives som en sluttspurt. En leser bør kunne forstå hvordan applikasjonen er bygget opp, hvordan du har modellert dataene, og hvordan du har skilt mellom det publikum får se og det redaktøren får tilgang til. Vurderinger rundt personvern, tilgangskontroll og hva som skjer hvis noe går galt hører naturlig hjemme her. Medlemsregisteret inneholder personopplysninger, så det fortjener et eget avsnitt.

4. Opsjoner

Hvis du har tid, er disse fine ting å bygge videre med. De er ikke krav.

- Egne sider per aktivitet med beskrivelse, kontaktperson, bilder og treningstider.
- Flere roller, for eksempel en egen kassererrolle som bare ser medlemsregisteret, eller gruppeledere som bare kan publisere innhold for sin egen aktivitet.
- Filtrering og søk i artikler og arrangementer.
- Begrenset antall plasser per arrangement med håndtering av venteliste.
- Eksport av medlemsregisteret til CSV.
- iCal eksport av kalenderen slik at medlemmer kan abonnere på den i sin egen kalenderapp.
- Bildebibliotek eller galleri.
- Påminnelser ved forfalt kontingent, for eksempel som e post eller varsling i admin.
- En logg over hvem som har endret hva og når i redaktørgrensesnittet.
- En enkel statistikkside for redaktøren, for eksempel mest leste artikler eller antall påmeldte per arrangement.

- Mørk modus.

5. Arbeidsform

Planlegg arbeidet med brukerhistorier og oppgaver i Jira eller et tilsvarende verktøy. Du velger selv hvilken metodikk du følger, og forklarer kort hvorfor. Skriv dokumentasjonen mens du jobber.

Planlegging

Min tolkning av oppgaven

Etter hvordan jeg har forstått oppgaven virker den veldig overkommelig og den har mange muligheter for ekstra funksjonalitet i tillegg til kravene.

Utenom ser jeg ingen problemer per nå med tanke på planlegging og dokumentering underveis og jeg føler at dette kan bli godt gjennomført sammen med utviklingen. Et av kravene til tjenesten er at koden og selve tjenesten skal dokumenteres. Dette er et viktig punkt hvis tjenesten er ment til å bli brukt over lengre tid og nye utviklere skal inn i kodebasen.

Med tanke på hvor kort tid jeg har, må jeg ta en vurdering på hvilke eventuelle ekstra funksjoner jeg har tid til å legge inn, og om de kan bli implementert på en god måte sammen med hovedfunksjonene til tjenesten. Av hva jeg kan vurdere nå, virker det som at en del av de ekstra funksjonene som blir nevnt kan bli med i løsningen uten å gå over tiden.

Jeg ser nå at noe ekstra funksjonalitet, som filtrering og søk i artikler og arrangementer, er noe jeg burde prioritere å legge til. Siden klubben har mange forskjellige idretter, vil en felles liste med alle arrangementer fort bli veldig uoversiktlig som den Facebook-løsningen de har i dag. Ved å legge til funksjonalitet for søk og filtrering løser vi et av de største problemene klubben har i dag.

Samtidig må jeg passe på og gjøre en risikovurdering for at de andre litt mer komplekse funksjonene. Funksjonalitet som kalenderekspert, bildegalleri og endringslogg virker ganske teknisk krevende og kan ta tid bort fra feilsøking og testing. Jeg kommer nok derfor til å nedprioritere ekstra features som disse, men hvis jeg ser at ting går bra og jeg får ekstra tid kan det å se på ting som å legge til en ekstra rolle for kasserer eller egne sider per aktivitet være en god utvidelse av tjenesten.

Min tolkning av tjenesten

Jeg skal bygge en webapplikasjon for Bekkelaget IL. Applikasjonen skal hovedsakelig bestå av to deler:

1. En åpen publikumsdel der brukere enkelt kan navigere seg gjennom nyheter, artikler og melde seg på arrangementer.
2. En lukket administrator del der redaktører fra klubben har mulighet til å opprette, slette og redigere arrangementer, nyheter, artikler og lignende, samt administrere medlemsregisteret.

Jeg skal bygge webapplikasjonen med redaktørene og brukerne i tankene. Jeg vil sette meg i deres sko for å se hva som er viktig for en vanlig bruker som kjapt vil finne informasjon om arrangementer eller nyheter og melde seg på, og for redaktører som ønsker en enkel måte å styre innholdet på uten at det blir for teknisk komplisert.

Hvorfor klubben trenger dette akkurat nå?

Slik situasjonen er i dag, bruker Bekkelaget IL en Facebook-side som hovedkanal. Problemet med dette er at viktige innlegg fort drukner i feeden til folk, og det er veldig vanskelig for foreldre og medlemmer å finne fram til kommende arrangementer eller treninger. I tillegg styrer de medlemsregisteret og kontingenter manuelt gjennom Excel-filer som sendes rundt på e-post. Dette gjør det uoversiktlig, det er lett å gjøre feil, og det er ikke en spesielt sikker måte å behandle personopplysninger på.

2. Krav til applikasjonen

Applikasjonen må oppfylle kravene som idrettslaget har behov for. Jeg har delt inn disse kravene i hva den vanlige brukeren skal kunne gjøre på nettsiden, og hva redaktørene skal kunne gjøre i redaktør-panelet.

Den åpne publikumsdelen

Forside: Ha en fin og oversiktlig forside som tydeliggjør hva klubben er, hva den står for og hva som skjer.

Nyheter og artikler: Siden skal ha en oversikt over artikler og nyheter slik at brukere slipper å lete på Facebook for å finne ut hva som skjer i klubben.

Se kommende arrangementer: Det skal være en oversikt over kommende arrangementer, dette kan være ting som treninger, kamper, dugnader.

Påmelding: Brukere må kunne trykke på en knapp for å melde seg på et arrangement direkte fra nettsiden, uten å måtte logge inn først.

Det lukkede redaktør panelet

Innlogging: Bare personer som er godkjent av klubben skal kunne logge inn i redaktørpanelet.

Styre innhold: Når man er logget inn som redaktør, skal man enkelt kunne opprette nye artikler, nyheter og arrangementer, redigere det som allerede ligger der, eller slette ting som er utdatert.

Innsyn til arrangementer: redaktører skal kunne ha oversikt over hvem som har meldt seg på et arrangement.

Styre medlemsregisteret: Redaktørene må kunne se en liste over alle medlemmene i klubben,

legge til nye medlemmer, og holde styr på hvem som har betalt kontingenten sin.

Brukerstyring: Admins skal kunne legge til redaktører og andre admins.

Oppgaven nevner terminlister i introduksjonen, men det står ikke oppført som et direkte krav i selve løsningen. Jeg kommer til å løse dette med å bruke oversikten over arrangementer, der man kan filtrere på kategorier slik at brukere enkelt kan finne de arrangementene som gjelder kamper og treninger for sin idrett. På denne måten dekker vi klubbens behov ved å bruke det systemet vi allerede har, uten å gjøre databasen unødvendig komplisert.

Applikasjonen skal være responsiv, følge lover og regelverk for universell utforming og WCAG, og gi en god brukeropplevelse på både mobil og desktop. Koden skal være av god kvalitet, typesikker (TypeScript) og lett å vedlikeholde. Jeg står fritt til å velge verktøy.

Prosjektet skal planlegges med brukerhistorier og tasks i Jira eller tilsvarende verktøy. Kildekoden skal være tilgjengelig i Git/Github gjennom hele perioden, og den ferdige løsningen skal være deployet og tilgjengelig på nett. Dokumentasjonen skal svare på hvorfor valg av metodikk og teknisk arkitektur er gjort.

2.1. Brukerhistorier

For å sikre at løsningen møter brukernes behov, har jeg definert følgende kjerne-historier:

For vanlige besøkende og foreldre:

- **Som en bruker** ønsker jeg å se en oversikt over nyheter på forsiden, slik at jeg enkelt kan finne ut hva som skjer i klubben uten å måtte lete på Facebook.
- **Som en bruker** ønsker jeg å se en oversikt over artikler og kommende arrangementer, slik at jeg vet når og hvor det er treninger, kamper eller dugnader.
- **Som en bruker** ønsker jeg å kunne melde meg på et arrangement med en enkel knapp, uten at jeg må logge inn eller opprette en egen bruker først.

For redaktører og administratorer:

- **Som redaktør** ønsker jeg å kunne logge meg inn på en sikker måte, slik at uvedkommende ikke får tilgang til klubbens interne verktøy og data.
- **Som redaktør** ønsker jeg å kunne opprette, redigere og slette nyheter, artikler og arrangementer fra et enkelt kontrollpanel, slik at innholdet på nettsiden alltid er oppdatert.
- **Som redaktør** ønsker jeg å se en oversikt over hvem som har meldt seg på de ulike arrangementene, slik at klubben har kontroll på antall deltakere.
- **Som redaktør** ønsker jeg å ha en digital medlemsliste med oversikt over hvem som har betalt kontingenten, slik at vi kan kaste de gamle Excel-arkene og lagre personopplysninger på en sikker måte.
- **Som Admin** vil jeg kunne legge til og fjerne redaktører og admins.

3. Generelt

Oppgaven består hovedsakelig av 4 deler, planlegging, gjennomføring, dokumentasjon og egenvurdering.

4. Generelle tanker om oppgaven

Jeg tenker i utgangspunktet at oppgaven er veldig grei og skal kunne løses godt, men jeg ser at det er noen fallgruver jeg må være obs på. Spesielt gjelder det med autentisering og tilgangskontroll, og hvordan jeg skal sette opp medlemsregisteret.

Siden siden skal brukes av forskjellige mennesker, planlegger jeg å løse tilgangen ved å bruke roller i databasen. De to rollene jeg tenker er redaktør og admin. Redaktører må kunne opprette, slette og redigere innhold og se medlemsregisteret, mens admin i tillegg skal kunne legge til eller fjerne andre redaktører. Vanlige brukere som besøker nettsiden trenger ingen innlogging eller rolle, da de bare skal kunne se på siden og melde seg på oppkommende arrangementer.

Når det gjelder selve medlemsregisteret, er det veldig viktig at jeg tenker nøye gjennom personvernet. Registeret inneholder personopplysninger, og for å gjøre det trygt vil jeg bruke prinsippet om dataminimering. Det betyr at jeg bare skal lagre akkurat den informasjonen klubben faktisk trenger for å erstatte de gamle Excel-arkene, som for eksempel navn, kontakinfo og om kontingenten er betalt. Jeg skal ikke samle inn noe mer enn det.

Dette med tilgangsstyring og personvern er punkter jeg risikerer å sette meg fast i hvis jeg gjør det for vanskelig. Jeg må tenke godt gjennom logikken, for eksempel ved å bruke RLS direkte i databasen for å låse medlemslistene. På den måten sikrer jeg at uautoriserte personer ikke kan hente ut data, samtidig som jeg ikke overkompliserer koden i frontenden og mister tid til selve "MVP-en".

Alt i alt er jeg veldig spent på å se hvordan jeg løser disse utfordringene i praksis.

Techstack

Frontend

I frontend planlegger jeg å bruke Next.js med TypeScript. Jeg har mye erfaring med dette og det er det rammeverket jeg kjenner best. Next.js er et stort og velkjent rammeverk som brukes i alt fra små prosjekter til store bedriftsløsninger. Det er veldig godt dokumentert, noe som gjør det lett å finne gode eksempler og løsninger på utfordringer som dukker opp.

Jeg velger Next.js fordi det gir applikasjonen veldig god ytelse. Ved å bruke "Server Components" kan jeg hente data fra databasen på serveren før siden sendes til brukeren. Dette gjør at siden laster raskt og fungerer godt for å hente ut arrangementer, nyheter og artikler. TypeScript bruker jeg gjennom hele prosjektet for å ha bedre kontroll på koden og unngå unødvendige feil, noe som er et krav i oppgaven.

Sammen med Next.js bruker jeg en del pakker jeg er godt kjent med og har god erfaring med å bruke:

- [Tailwind CSS](#): Dette bruker jeg til styling. Det lar meg bygge et responsivt design raskt og effektivt. Siden oppgaven krever at løsningen skal fungere godt på mobil, er Tailwind et godt valg da det gjør det enkelt å styre hvordan elementer endrer seg på ulike skjermstørrelser.
- [Shadcn/UI](#): Dette er et komponentbibliotek jeg kommer til å bruke for å bygge applikasjonen. Ferdiglagde komponenter for knapper, kort, inputs osv. kommer til å øke hastigheten jeg kan lage UI-en på, slik at jeg kan bruke mer tid til å fokusere på selve brukerflyten og logikken.
- [Lucide React](#): Jeg bruker Lucide for ikoner. Gode ikoner gjør applikasjonen mer intuitiv og lettere å navigere i for brukeren.
- [BiomeJs](#): Jeg bruker Biome til linting og formatering av koden. Det hjelper meg med å holde kodebasen ryddig, fjerne ubrukt kode og sikre at jeg skriver god og vedlikeholdbar kode gjennom hele prosjektet. Det er også mye raskere enn eldre verktøy som ESLint og Prettier, noe som gir en bedre flyt i utviklingen.
- [Zod](#): Brukes for å håndtere skjemavalidering. Dette sikrer at dataene redaktører legger inn er korrekte, og gir brukeren gode feilmeldinger i sanntid.
- [Date-fns](#): Siden håndtering av tid og dato vil være sentralt i løsningen for arrangementer og nyheter, brukes date-fns for å formatere og vise tidspunkter på en brukervennlig måte.
- [Sonner \(Toasts\)](#): Dette er en del av Shadcn/UI som jeg bruker til å gi tilbakemeldinger til brukerne. Når en besøkende melder seg på et arrangement, eller en redaktør oppretter en nyhet, vil det dukke opp en liten varsling på skjermen som bekrefter at handlingen var vellykket.

Backend og Database

Som backend og database har jeg valgt å bruke Supabase. Dette er en "Backend-as-a-Service" (BaaS) som bygger på en PostgreSQL-database. Jeg har valgt denne teknologien fordi den lar meg sette opp en profesjonell og sikker backend veldig raskt, slik at jeg kan bruke mer tid på å utvikle funksjonaliteten til applikasjonen.

Jeg bruker Supabase til følgende kjerneoppgaver:

- **Database (PostgreSQL)**: Dataene til idrettslaget henger veldig tett sammen. For eksempel skal en påmelding på et arrangement være knyttet til en spesifikk person, og en nyhetsartikkel skal være koblet til redaktøren som skrev den. Siden alt har en klar sammenheng, er en relasjonsdatabase det beste valget for å holde dataene ryddige og sikre god dataintegritet.
- **Autentisering (Supabase Auth)**: Jeg bruker denne tjenesten for å håndtere innlogging for klubbens redaktører og administratorer. Dette er en sikker løsning som følger

bransjestandarder. Systemet gjør at passordene blir kryptert automatisk, slik at jeg som utvikler aldri har tilgang til eller lagrer passordene.

- **Row Level Security (RLS):** Dette er en innebygd sikkerhetsfunksjon i databasen som jeg bruker for å kontrollere hvem som har lov til å se, endre eller slette data. Dette er en veldig sikker måte å håndtere tilgangskontroll på, da reglene ligger helt nede i databasen. Jeg setter strenge regler som gjør at kun innloggede redaktører får lov til å se eller endre på medlemslistene, noe som er kritisk for å beskytte personopplysningene i medlemsregisteret.

Vurdering av arkitektur og serverlogikk

Jeg har også vurdert å bruke Supabase Edge Functions for serverlogikk, men har valgt å la være for denne oppgaven. Siden jeg bruker Next.js, kan jeg håndtere server side logikk og database kall direkte gjennom Server Actions og API Routes. Dette holder kodebasen mer samlet, gjør arkitekturen enklere å vedlikeholde, og er mer enn nok for å dekke kravene til en god MVP for idrettslaget.

Drøfting av autentiseringsmetode

For redaktør-panelet har jeg valgt å gå for e-post og passord som hovedmetode for innlogging. Siden det kun er et mindre antall redaktører og administratorer i klubben som skal logge inn i redaktør-panelet, er dette en trygg metode som de fleste kjenner godt til som krever lite opplæring. Vanlige brukere på nettsiden skal kunne gå til arrangementer og melde seg på uten å logge inn. Jeg kommer til å løse dette med å bruke navn og e-post/telefonnummer for påmeldingen slik at vi gjør det så enkelt som mulig for brukerne.

Verktøy/applikasjoner

For å ha god kontroll på prosjektet og sikre en strukturert arbeidsprosess, vil jeg bruke følgende verktøy:

- [Jira](#): For prosjektstyring, oppretting av brukerhistorier og sporing av oppgaver.
- [Git](#) & [GitHub](#): For versjonskontroll og lagring av kildekode.
- [Cursor](#): Min foretrukne kodeeditor for selve programmeringen.
- [Slack](#), [Teams](#) & [Outlook](#): For kommunikasjon med kollegaer, veiledere og sensorer.
- [Dbdiagram](#): For visualisering av database tabeller og koblinger
- [Supabase Dashboard](#): For administrasjon av database, autentisering og lagring.
- [Gemini](#): Brukes som støtteverktøy for ideutvikling og som assistent og sparringspartner.
- [Vercel](#): For hosting. Jeg bruker Vercel for å møte kravet i oppgaven om at applikasjonen skal være deployet og tilgjengelig på nett.

Autentisering

For å holde applikasjonen så enkel og oversiktlig som mulig, deler jeg brukerne inn i tre tydelige nivåer. Dette sikrer god tilgangskontroll og hindrer at sensitive data kommer på avveie:

- **Vanlig bruker (Uinnlogget):** Dette er den vanlige brukeren av nettsiden. De trenger ikke å opprette bruker eller logge inn. De har utelukkende lesetilgang til forsiden, nyheter, artikler og arrangementer, og de kan melde seg på arrangementer uten innlogging.
- **Redaktør (Innlogget):** Dette er godkjente personer fra klubben som har logget inn med e-post og passord. De har tilgang til redaktør-panelet der de kan opprette, redigere og slette nyheter, artikler og arrangementer, se hvem som er påmeldt, og administrere medlemsregisteret.
- **Administrator (Inlogget):** Admin-rollen har det øverste ansvaret. I tillegg til å kunne gjøre alt en redaktør kan, har admin full tilgang til å opprette og invitere nye redaktører til systemet. Admin-rollen skal også settes opp med rettigheter i databasen som gjør at de kan overstyre de vanlige RLS-reglene, slik at de har full kontroll hvis noe må overskrives eller rettes opp i systemet.

Styring og opprettelse av redaktører

Når en ny redaktør skal legges til i klubben, skal det ikke være mulig for hvem som helst å registrere seg på nettsiden. Opprettelsen styres strengt fra innsiden av kontrollpanelet av en administrator og det skal følge denne flyten:

1. Admin logger seg inn og går til siden for brukerstyring i admin-panelet.
2. Admin skriver inn e-posten til den nye redaktøren og velger rollen **editor**.
3. Systemet oppretter brukeren i Supabase Auth og sender ut en invitasjon på e-post med en lenke der den nye redaktøren kan sette sitt eget passord.
4. Samtidig blir rollen lagret i en egen **profiles**-tabell i databasen, slik at systemet vet nøyaktig hvilke rettigheter denne e-posten har.

Dette sikrer en lukket og trygg prosess, slik at ingen utenfor Bekkelaget IL kan opprette en konto.

Implementasjon av sikkerhet

Sikkerheten i applikasjonen kommer til å bli håndtert på to uavhengige nivåer for å sikre at ingen uvedkommende får tilgang til klubbens interne data:

- **Sikkerhet i frontenden (Next.js Middleware):** Jeg bruker middleware til å beskytte alle ruter som starter med **/admin**. Hvis en anonym besøkende prøver å taste inn en slik nettadresse manuelt, sjekker systemet om brukeren har en aktiv sesjon og en gyldig rolle. For at redaktørene skal kunne logge inn uten å bli stoppet av middleware, legger jeg selve innloggingssiden på ruten **/login**. Når en redaktør har logget inn her, blir de automatisk videresendt inn til det lukkede redaktør-panelet. Hvis noen som ikke er logget inn prøver å snike seg inn på **/admin**, blir de automatisk kastet ut og videresendt til forsiden.

- **Sikkerhet i backenden (Row Level Security):** Siden frontendsikkerhet i teorien kan omgås ved å manipulere forespørsler direkte i nettleseren, legger jeg den viktigste sperren nede i databasen med å bruke RLS. For tabellen med medlemsregisteret setter jeg en streng regel (policy) som sier at kun autentiserte forespørsler med rollen **editor** eller **admin** får lov til å lese eller skrive data. Admin-rollen settes opp til å automatisk ha full tilgang (**ALL**) til alle tabeller, slik at de kan overskrive og rydde opp i systemet ved behov.

Sikkerhet og personvern

For å sikre ansvarlig behandling av personopplysninger i Bekkelaget IL, har jeg prøvd å ha GDPR og prinsippet om "Privacy by Design" i bakhodet gjennom hele planleggingen. Jeg føler jeg løser dette på en god måte med bruk av disse tiltakene.

- **Dataminimering:** Systemet samler kun inn nødvendige personopplysninger (navn, e-post, telefonnummer) for å administrere medlemskap og påmeldinger. Ingen unødvendige data lagres, i tråd med Datatilsynets retningslinjer.
- **Autentisering og passordhåndtering:** Jeg benytter Supabase Auth for sikker brukerautentisering. Passord blir automatisk kryptert med hashing, noe som betyr at jeg som utvikler aldri har tilgang til brukernes passord.
- **Tilgangsstyring:** Sikkerheten er bygget i to lag. Jeg bruker Next.js Middleware for å hindre uautorisert tilgang på klientsiden, og Row Level Security (RLS) i Supabase for å sikre at sensitive data kun er tilgjengelige for autentiserte brukere med riktig rolle, direkte i databasen.
- **Sikker deling:** Ved deling av miljøvariabler og sensitive nøkler under utvikling, kommer jeg til å benytte 1ty.me for å sikre at nøkler ikke kommer på avveie i e-post eller chat-historikk.

Database struktur

For å sikre god dataintegritet og effektiv henting av informasjon, har jeg valgt å bygge databasen opp som en relasjonell database i PostgreSQL med bruk av Supabase. Jeg har satt opp strukturen for å håndtere relasjoner mellom de ulike delene av systemet.

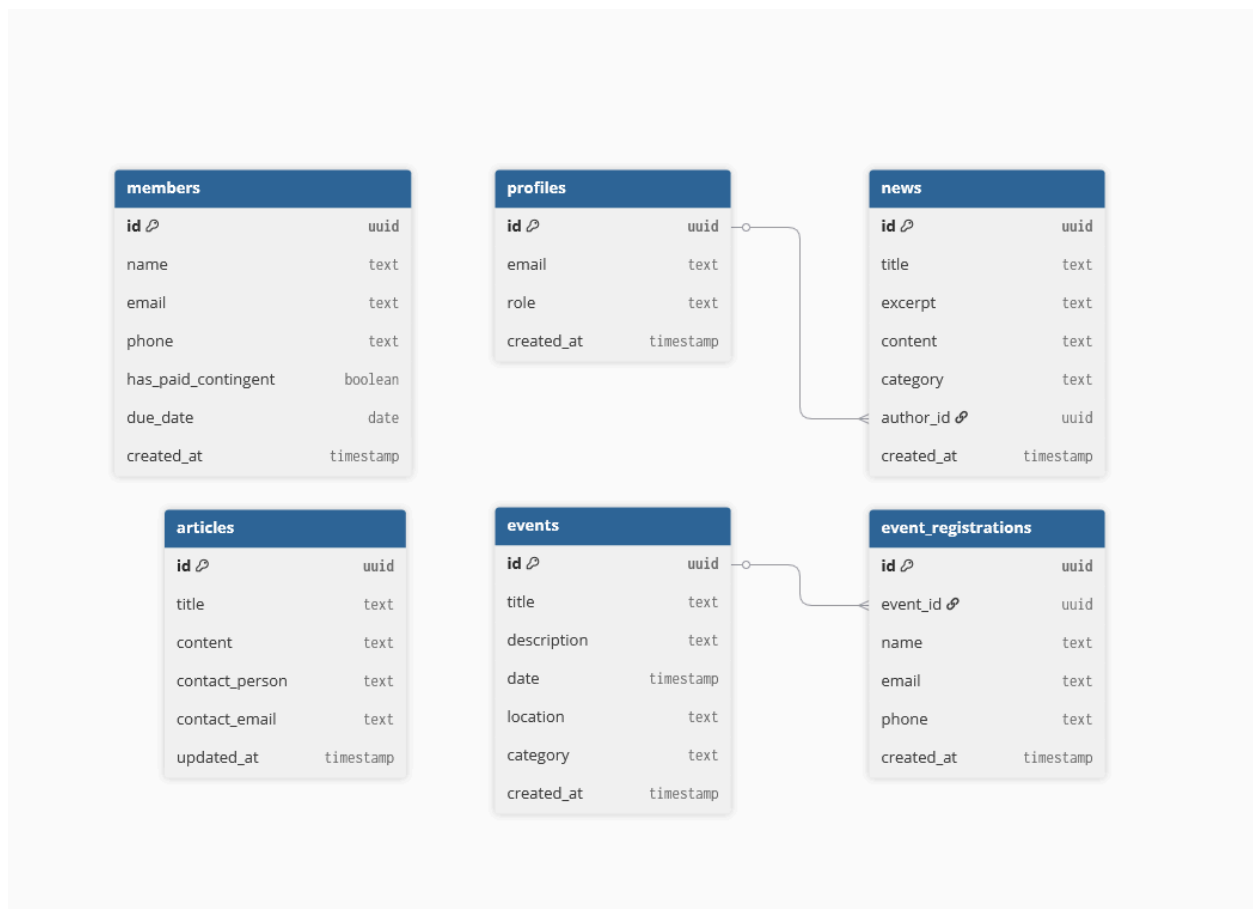
Tabelloversikt

Systemet består av følgende hovedtabeller:

- **profiles:** Inneholder utvidet informasjon om brukerne. Denne er koblet mot Supabase Auth sin interne brukertabell via en UUID.
- **news:** Inneholder dataene for nyhetene til klubben.
- **articles:** Inneholder dataene for artiklene til klubben.
- **events:** Inneholder dataene for arrangementene til klubben.
- **event_registrations:** Registrerer påmeldinger til arrangementene fra brukere.
- **members:** Klubbens lukkede medlemsregister med oversikt over kontaktinfo og kontingentstatus.

Entity Relationship Diagram (ERD)

Diagrammet under viser hvordan tabellene er koblet sammen



[Lenke til Diagram](#)

Relasjoner

For å få applikasjonen til å fungere som ønsket, er tabellene koblet sammen slik:

- **Profil til nyheter:** Hver nyhetsartikkel er koblet til en profil gjennom `author_id`. Dette gjør at vi alltid vet hvilken redaktør eller admin som har lagt ut nyheten.
- **Arrangement til Påmelding:** Hver påmelding er koblet direkte til et arrangement gjennom `event_id`. Dette gjør at systemet holder orden på hvem som er påmeldt til hvilken aktivitet, og vi har satt på cascade-sletting slik at påmeldingene forsvinner automatisk hvis et arrangement blir slettet.

Arbeidsmetodikk

For dette prosjektet følte jeg at det beste valg av arbeidsmetodikk vil være Agile metodikk, med hovedfokus på Kanban med bruk av Jira. Jeg valgte kanban med tanke på hvor lang tid jeg har til å utvikle løsningen, som gjør det vanskelig å følge andre arbeidsmetoder som Scrum eller Fosefall. Det gir mulighet til forbedringer og refleksjoner underveis i utviklingsprosessen som kan være nyttig med tanke på tid til både utvikling og planlegging som kan ha forårsaket at visse ting har blitt glemt i planleggingsfasen. Det hjelper også med å skape et kontinuerlig syn på framgang med board som vil gjøre det lett og følge med på tidsestimering av oppgaver som kanskje tar mer eller mindre tid enn forventet

Design og layout

Før jeg går fram med utviklingen har jeg også prøvd å planlegge siden med hvordan jeg vil at den generelle layouten av sidene skal være og hva slags farger som vil passe. Siden Bekkelaget IL i denne oppgaven er en breddeklubb basert i Ottestad, har jeg valgt å definere en egen fargepalett for løsningen. Jeg har valgt å bruke gult å blått som hovedfarger, jeg synes denne kombinasjonen gir et veldig sportslig inntrykk som passer bra til en klubb med flere forskjellige sporter. Med å sette opp en fargepalett for laget vil løsningen få en mer tydelig identitet som brukere og medlemmer kan bli vant til. Her er fargepaletten:



Jeg brukte coolors.co for å lage fargepaletten

For å sikre et ryddig, sportslig og universelt utformet uttrykk, har jeg definert disse fargene som fargepaletten for løsningen.

- **Primary (#0F172A):** En dyp mørkeblå farge som brukes på hovedmenyer, rammer og store overskrifter.

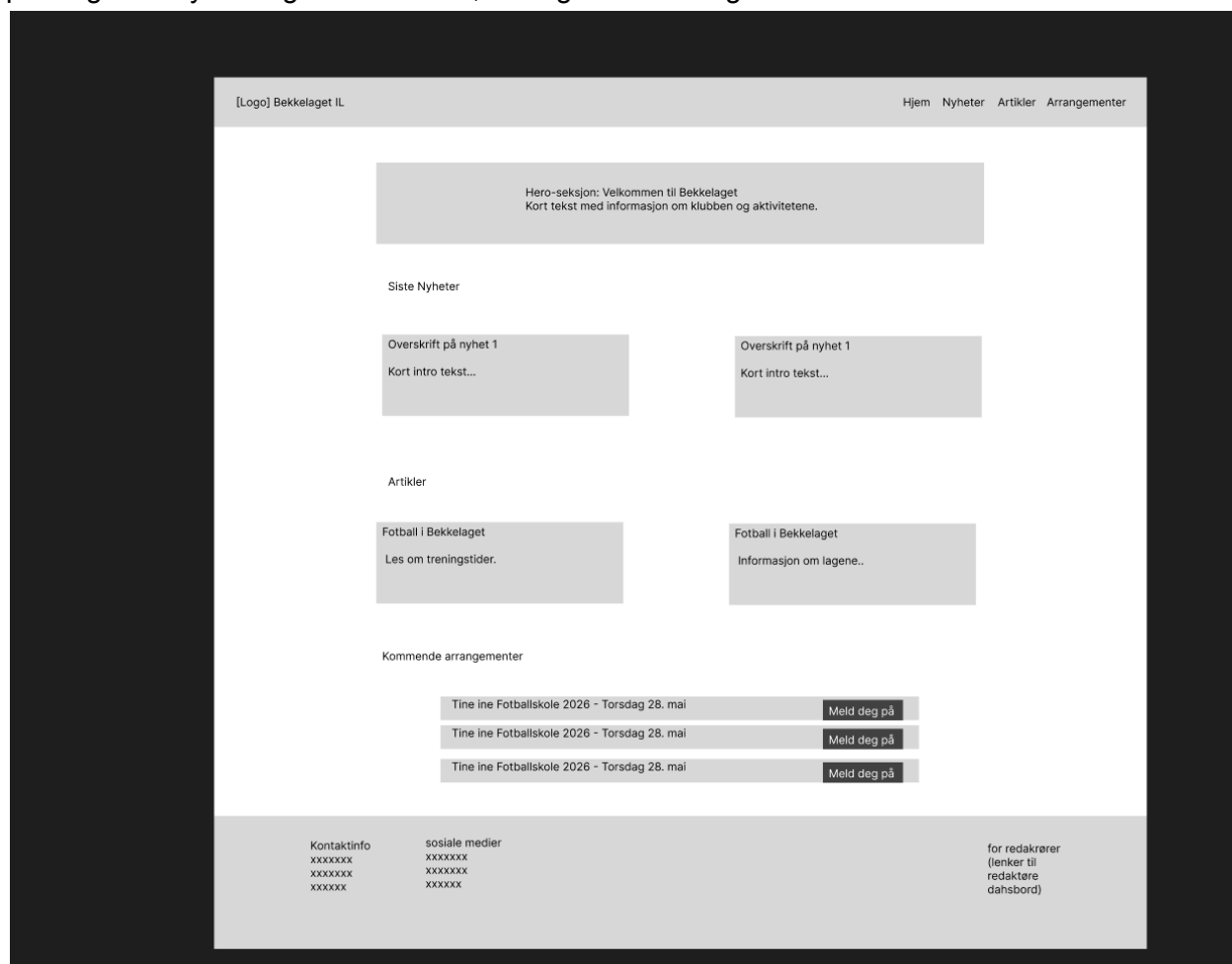
- **Accent (#EAB308):** En sportslig gulfarge som brukes som en "action-farge" på viktige knapper som "Meld deg på" for å fange brukerens oppmerksomhet.
- **Background / Card (#F8FAFC / #E2E8F0):** Lyse, nøytrale toner som sørger for at forsiden, nyhetsfeeden og tabellene er rene og behagelige å lese.
- **Success / Destructive (#22C55E / #EF4444):** Grønn og rød brukes kun til funksjonelle ting i medlemsregisteret, og påmelding som å vise om en kontingent er betalt eller Ikke betalt og på sletteknapper.

For å være sikker på at løsningen oppfyller lovkravene til universell utforming, har jeg kjørt fargekombinasjonene gjennom en WCAG-kontrastsjekker under planleggingen. Jeg brukte whocanuse.com for å se hvordan fargene passer.

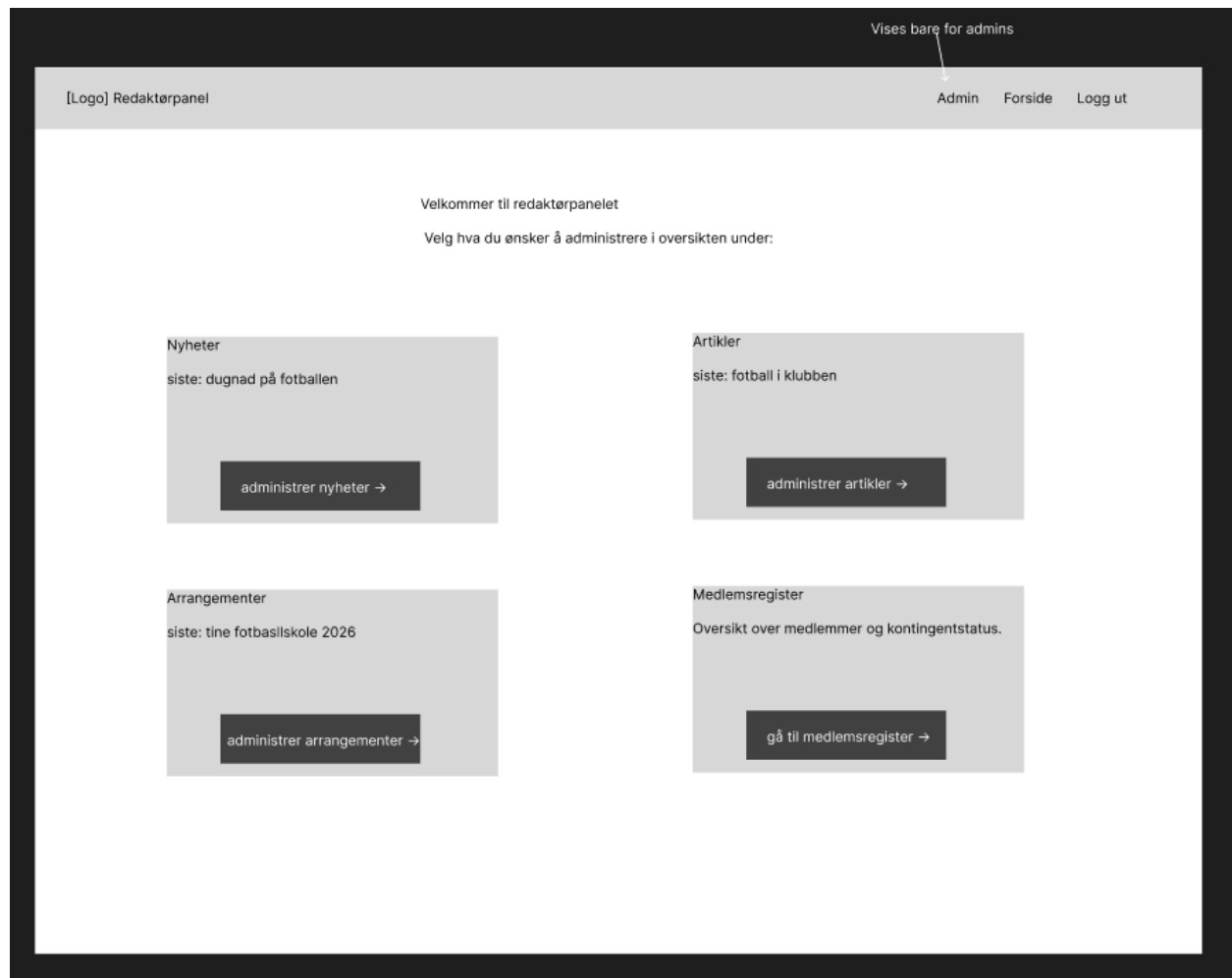
- Den mørkeblå bakgrunnen (#0F172A) med hvit tekst gir en kontrast på over 17:1 og er godt innenfor AAA-kravet.
- Siden gult (#EAB308) har lav kontrast mot hvit tekst, har jeg tatt et bevisst designvalg om å bruke den mørkeblå fargen som tekstfarge inni de gule knappene og komponentene. Dette gjør at også de gule elementene oppfyller kontrastkravene (AA/AAA) og blir enkle å lese for svaksynte.

For å finne ut hvordan innholdet burde ligge i siden før jeg begynner med utviklingen har jeg også tegnet opp en veldig enkel layout i figma. Her har jeg bare skissert opp plasseringen til et

par ting som nyhets og artikelfeeden, arrangementene og redaktør dashboard.



Skisse hjemmeside



Skisse redaktørpanel

Dag til dag plan

Tirsdag 26. Mai – Dag 1

Jeg bruker den første dagen til å sette meg godt inn i oppgaveteksten og definere min tolkning av oppdraget. Jeg vil kartlegge all funksjonaliteten som kreves for Bekkelaget IL, og skrive ned brukerhistoriene som danner grunnlaget for applikasjonen.

Onsdag 27. Mai – Dag 2

Dagen brukes til å ferdigstille planleggingsfasen og utarbeide en dagsplan for hele prosjektløpet. Bryte ned arbeidet i konkrete oppgaver og putte de inn i Jira for hver av dagene

slik at alt ligger til rette for en effektiv utviklingsfase på torsdag. Hvis jeg får tid begynner jeg på oppsett av Github-repo og Supabase-prosjekt, initialisere Next.js-prosjektet og koble sammen frontend og backenden.

Torsdag 28. Mai – Dag 3

Jeg starter utviklingen med å få på plass innloggingen og den rollebaserte tilgangen via Supabase. Når det er i orden, fokuserer jeg 100 % på administrasjonssidene. Jeg vil lage skjemaene og funksjonaliteten for å opprette, redigere og slette nyheter, artikler og arrangementer. Målet er å få alt av baksidesystemer, input validering og dataflyt til å fungere hundre prosent først.

Fredag 29. Mai – Dag 4

Lage det lukkede medlemsregisteret for styret med en ryddig oversikt over alle medlemmene i klubben. Legge til funksjonalitet for å søke etter personer i listen og en enkel måte å endre status på hvem som har betalt eller ikke betalt klubbkontingenten. Målet er at hele kjernesystemet for admin og redaktører skal være helt ferdig før helgen og kanskje få begynt på det brukerne skal se.

Mandag 1. Juni – Dag 5

Nå som alt det tekniske i bakgrunnen fungerer og dataene lagres riktig, flytter jeg fokuset over på forsiden og det de vanlige besøkende skal se. Jeg lager hovedsiden med sider til nyheter, arrangementer og artikler, og setter opp påmeldingsskjemaet for arrangementer.

Tirsdag 2. Juni – Dag 6

Hele denne dagen blir brukt til å gå grundig gjennom applikasjonen for å feilsøke og teste at alt fungerer som det skal, muligens jobbe på ekstra features hvis jeg har nok tid. Jeg vil også sjekke at løsningen oppfyller kravene til universell utforming og WCAG, og at sikkerheten i databasen er tett. Når alt er testet, bruker jeg resten av dagen til å begynne på de tekniske delene av dokumentasjonen.

Onsdag 3. Juni – Dag 7

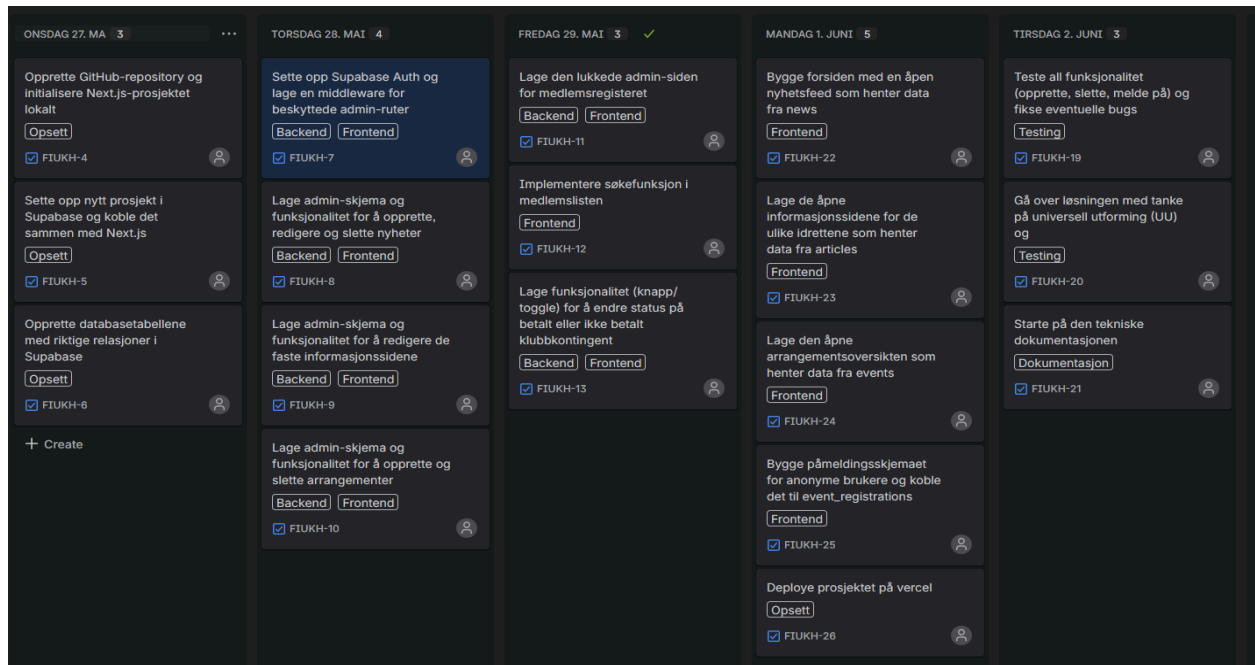
Gjøre ferdig dokumentasjonen og egenvurderingen. Forberede en demo av nettsiden og en gjennomgang av hvordan jeg har jobbet i Jira i løpet av perioden. Levere inn koden og dokumentasjonen. Presentere prosjektet for sensorene, svare på spørsmål om løsningene mine og forhåpentligvis stå 😊

Gjennomføring

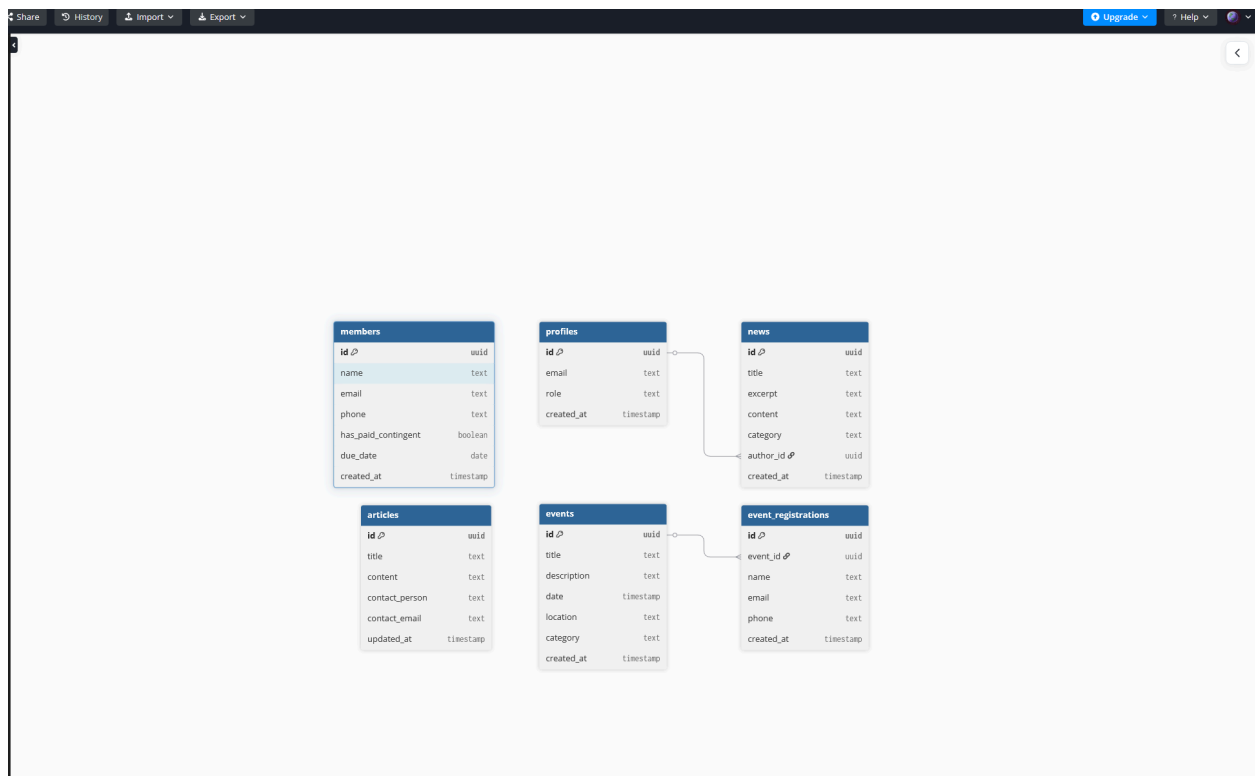
Tirsdag 26. Mai – Dag 1

Dagen startet med at jeg fikk utdelt fagprøven. Jeg hadde et oppstartsmøte med prøvenemnda hvor vi pratet om oppgaven og forventninger fra begge parter. Vi ble enige om hvordan vi skulle sette opp innleveringen av planleggingen og statusmøtene de kommende dagene.

Etter møtet begynte jeg med planleggingen av fagprøven. Jeg startet med å skrive min tolkning av oppgaven. Deretter satte jeg opp et Jira board for å strukturere arbeidsoppgavene for resten av prøveperioden. Jeg begynte også å skisse opp backend-strukturen ved bruk av dbdiagram.io for å visualisere databasemodellen.



Jira board



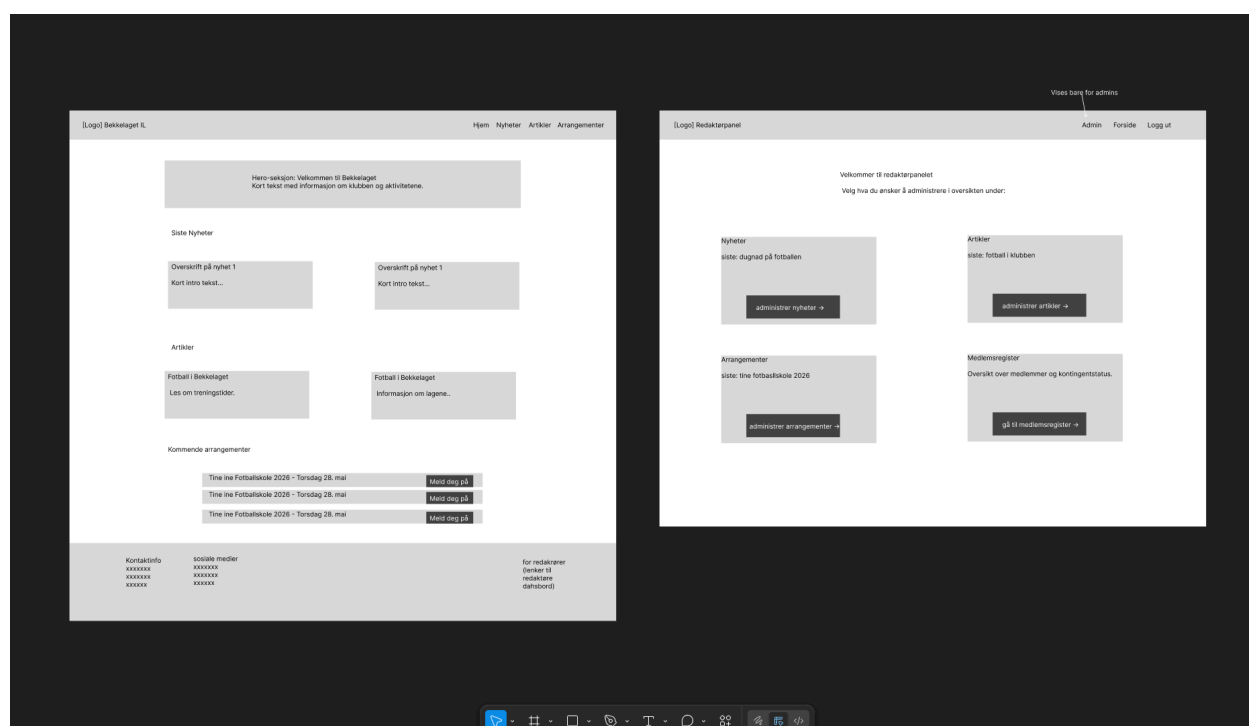
Database diagram

Etter at jeg hadde skissert opp databasestrukturen, begynte jeg å definere tech-stacken. Jeg vurderte kravene til prosjektet og valgte verktøy som jeg synes fungerer godt sammen for å

bygge en løsning som er enkel å vedlikeholde. Mot slutten av dagen gjorde jeg ferdig beskrivelsen av arbeidsmetodikken min (agile) og begynte på en dagsplan for resten av uken.

Onsdag 27. Mai – Dag 2

Dagen gikk for det meste bort til å ferdigstille planleggingen. Jeg skrev ferdig dag til dag planen for resten av uken, slik at jeg har kontroll over hva jeg burde fokusere på de forskjellige dagene. Jeg tok også en del ut av dagen for å tenke på design og fargebruk i løsningen. Jeg definerte en egen fargepalett for siden og skisserte opp noen røffe tegninger av hvordan jeg tenker at layouten av hovedsiden og redaktør dashboardet burde se ut, og tror dette vil hjelpe meg veldig i utviklingsprosessen med hvordan jeg skal bygge opp og designe sidene.



Figma skisse

Jeg møtte på noen utfordringer når det kom til design og skissen, knyttet til universell utforming og kontraster. Jeg hadde et første utkast av fargepaletten som jeg følte meg ganske fornøyd med, men når jeg sjekket fargene mot kravene for universell utforming med å bruke [whocanuse.com](https://www.whocanuse.com/), oppdaget jeg at de lyse bakgrunns og kortfargene matchet veldig dårlig med

hvit tekst.

whoCanUse

The quick brown fox jumps over the lazy dog

BACKGROUND #E2EF80 TEXT #FFFFFF 16 PX BOLD

If you'd like to help keep WhoCanUse maintained, please consider turning off your AdBlocker for us (it's cool if you don't want to, no pressure)

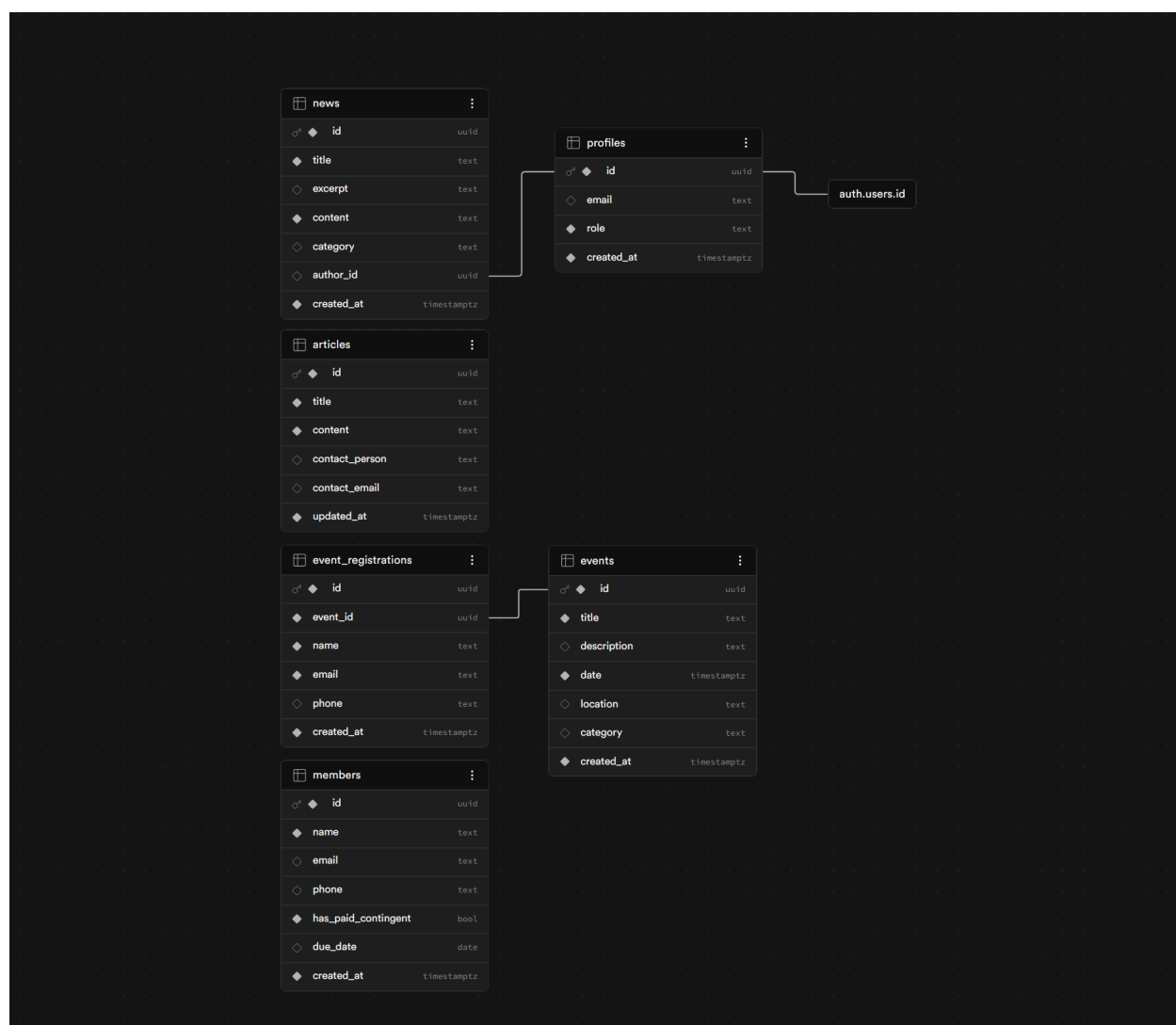
Who can use this color combination?

Contrast Ratio	1.23:1	WCAG Grading	FAIL
Regular Vision (Trichromatic) 🦋 Can distinguish all three primary color, little to no blindness	What I see	68% affected	
Protanomaly 🦋 Reduced sensitivity to red - trouble distinguishing reds and greens	What I see	1.3% affected	
Protanopia 🦋 Red blind - Can't see reds at all	What I see	1.5% affected	
Deuteranomaly 🦋 Reduced sensitivity to green - Trouble distinguishing reds and greens	What I see	5.3% affected	
Deuteranopia 🦋 Green blind - Can't see greens at all	What I see	1.2% affected	
Tritanomaly 🦋 Trouble distinguishing blues and greens, and yellows and reds	What I see	0.02% affected	
Tritanopia 🦋 Unable to distinguish between blues and greens, purples and red, and yellows and pinks	What I see	0.03% affected	
Achromatomaly 🦋 Partial color blindness, sees the absence of most colors	What I see	0.09% affected	
Achromatopsia 🦋 Complete color blindness, can only see shades	What I see	0.05% affected	
Cataracts 🦋 Clouding of the lens in the eye that affects vision	What I see	33% affected	
Glaucoma 🦋 Blind spots	What I see		

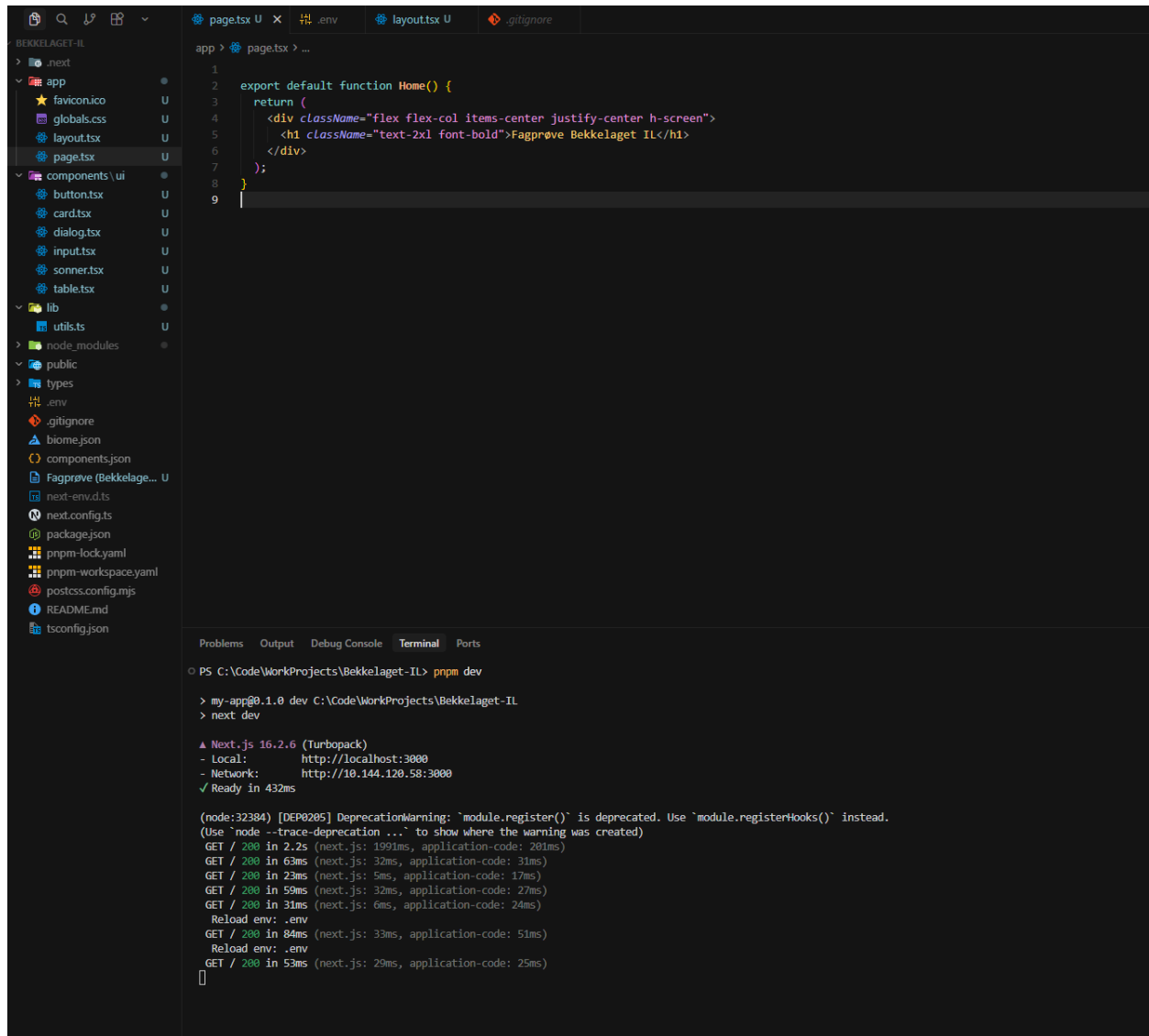
For å løse dette, valgte jeg å bruke en mørkegrå tekstfarge på de lyse flatene, og mørkeblå tekst inni de gule knappene. På denne måten sikret jeg at kravene til kontrast blir møtt.

Etter dette fikk jeg sendt planleggingen over til prøvenemnda og skal prate med dem imorgen om planen.

Det siste jeg gjorde var å opprette et repo på Github og initialisere prosjektet med alle pakkene jeg trenger og opprette prosjektet i Supabas og legge til alle tabeller i databasen.



Supabase dashboardet



```
1
2 export default function Home() {
3   return (
4     <div className="flex flex-col items-center justify-center h-screen">
5       <h1 className="text-2xl font-bold">Fagprøve Bekkelaget IL</h1>
6     </div>
7   );
8 }
9
```

```
PS C:\Code\WorkProjects\Bekkelaget-IL> pnpm dev
> my-app@0.1.0 dev C:\Code\WorkProjects\Bekkelaget-IL
> next dev

▲ Next.js 16.2.6 (Turbopack)
- Local:    http://localhost:3000
- Network:  http://10.144.120.58:3000
✓ Ready in 432ms

(node:32384) [DEP0205] DeprecationWarning: `module.register()` is deprecated. Use `module.registerHooks()` instead.
(Use `node --trace-deprecation ...` to show where the warning was created)
GET / 200 in 2.2s (next.js: 1991ms, application-code: 201ms)
GET / 200 in 63ms (next.js: 32ms, application-code: 31ms)
GET / 200 in 23ms (next.js: 5ms, application-code: 17ms)
GET / 200 in 59ms (next.js: 32ms, application-code: 27ms)
GET / 200 in 31ms (next.js: 6ms, application-code: 24ms)
Reload env: .env
GET / 200 in 84ms (next.js: 33ms, application-code: 51ms)
Reload env: .env
GET / 200 in 53ms (next.js: 29ms, application-code: 25ms)
```

Prosjektet initialisert med [Next.js](#)

Torsdag 28. Mai – Dag 3

Startet dagen med et møte med prøvenemnda for å få godkjent planleggingen min. Jeg følte dette gikk veldig greit, men under møte ble jeg stilt noen spørsmål som fikk meg til å tenke litt på hvordan jeg har valgt å løse oppgaven. Jeg merker det at en løsning der alt redaktørene styrer skal administreres gjennom Supabase, kan ha vært en litt tungvint måte for å legge opp innholdsstyring. Å bruke et cms verktøy som Sanity kunne ha vært en god løsning for opprettelser av artikler, arrangementer og nyheter. Jeg tenkte på dette litt i starten av planleggingen da jeg prøvde å finne beste måte å løse oppgaven på, men valgte å bruke supabase siden jeg hadde lyst på roller i redaktør panelet og det ville vært vanskelig å sette opp i sanity og en medlemsoversikt vil være mye tyngre å bruke siden hvert av medlemmene måtte blitt lagret som et dokument og det ville tatt lang tid å søke filtrere gjennom som hadde ført til

en tyngre redaktør flyt.

Noen av tankene jeg sitter med etter møte er at en løsning kunne vært å bruke sanity for innholdstyring av nyheter, artikler og arrangementer, og heller lagt inn en kobling å hatt medlemsregisteret i Supabase å vise det i et eget view inne i cms-et. Dette er nok noe jeg burde vurdert litt nærmere når jeg skrev selve planleggingen, men jeg er fornøyd med planen min og tror jeg kan lage et godt produkt i løpet av de kommende dagene.

Etter møtet begynte jeg med å sette opp RLS polycier i Supabase for å sikre at alle tabeller i databasen var beskyttet.

Jeg satte også opp en kobling mellom **profiles**-tabellen og Supabase auth slik at når en ny bruker blir laget, får den automatisk en rad i **profiles** med editor rollen.

schema public

Filter tables and policies

articles

Disable RLS

Create policy

NAME

COMMAND

APPLIED TO

articles_editor_admin_crud

ALL

authenticated

event_registrations

Disable RLS

Create policy

NAME

COMMAND

APPLIED TO

registrations_editor_admin_read

SELECT

authenticated

registrations_public_insert

INSERT

anon

events

Disable RLS

Create policy

NAME

COMMAND

APPLIED TO

events_editor_admin_crud

ALL

authenticated

events_public_read

SELECT

anon

members

Disable RLS

Create policy

NAME

COMMAND

APPLIED TO

members_editor_admin_crud

ALL

authenticated

news

Disable RLS

Create policy

NAME

COMMAND

APPLIED TO

news_editor_admin_crud

ALL

authenticated

news_public_read

SELECT

anon

profiles

Disable RLS

Create policy

NAME

COMMAND

APPLIED TO

profiles_select_own

SELECT

authenticated

profiles_update_admin_only

UPDATE

authenticated

RLS polycier i Supabase

Name of function

handle_new_user

Name will also be used for the function name in postgres

Schema

schema public

Tables made in the table editor will be in 'public'

Arguments

Arguments can be referenced in the function body using either names or numbers.

No argument for this function

Definition

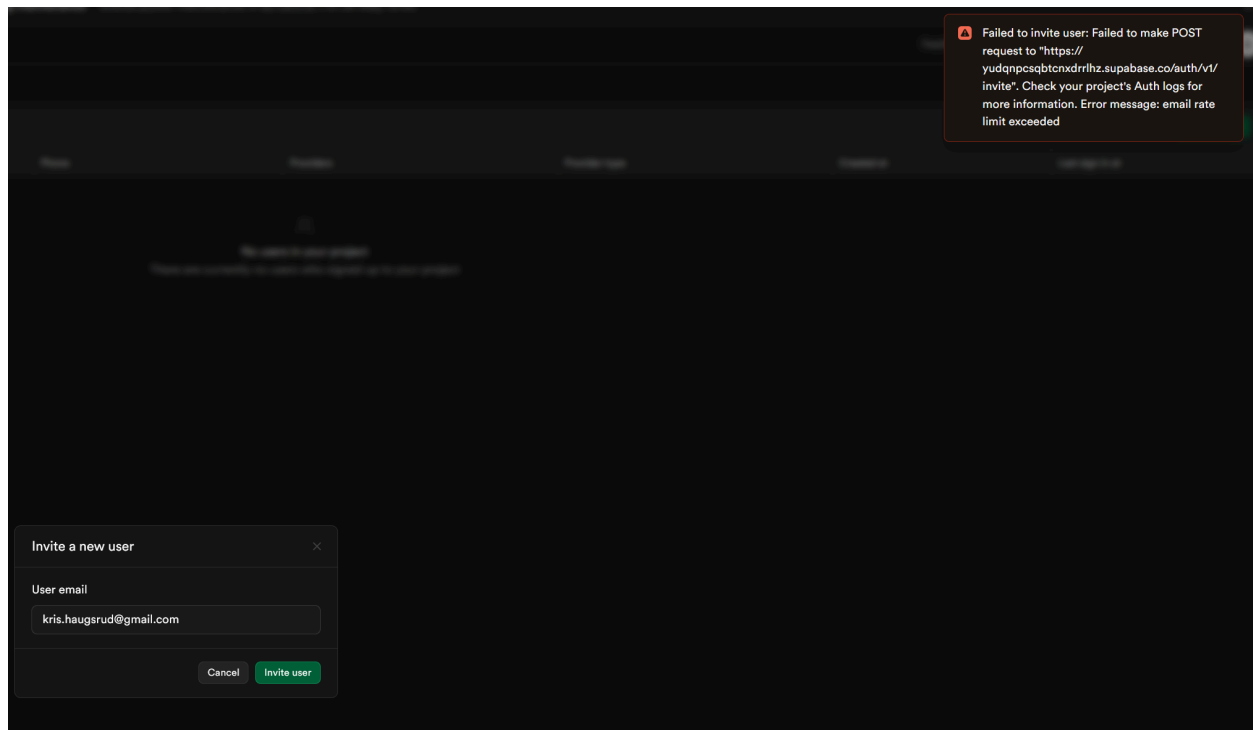
The language below should be written in plpgsql.

```
1
2 begin
3   insert into public.profiles (id, email, role, created_at)
4     values (new.id, new.email, 'editor', now())
5   on conflict (id) do update
6     set email = excluded.email;
7
8   return new;
9 end;
10
```

Funksjon som kobler Supabase auth med Profiles tabellen.

Etter dette begynte jeg å sette opp autentisering i prosjektet. Jeg satte opp login-sider og middleware som beskytter /admin-ruter, og laget en flyt der brukere skal kunne inviteres av admin og deretter sette passord. Under testing støtte jeg på en begrensning i Supabase (rate limit for utsending av e-post/invitasjoner). Jeg traff rate limiten etter å ha sendt en mail så det ble

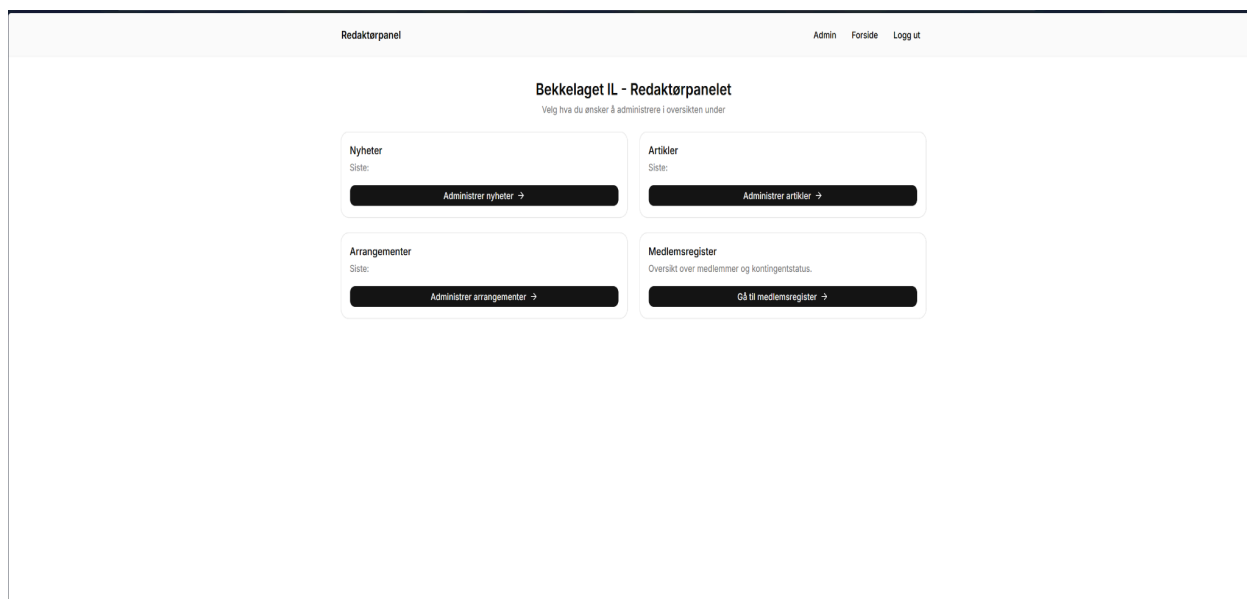
vanskelig å teste og verdifisere flyten av å opprette brukere.



Supabase rate limit error

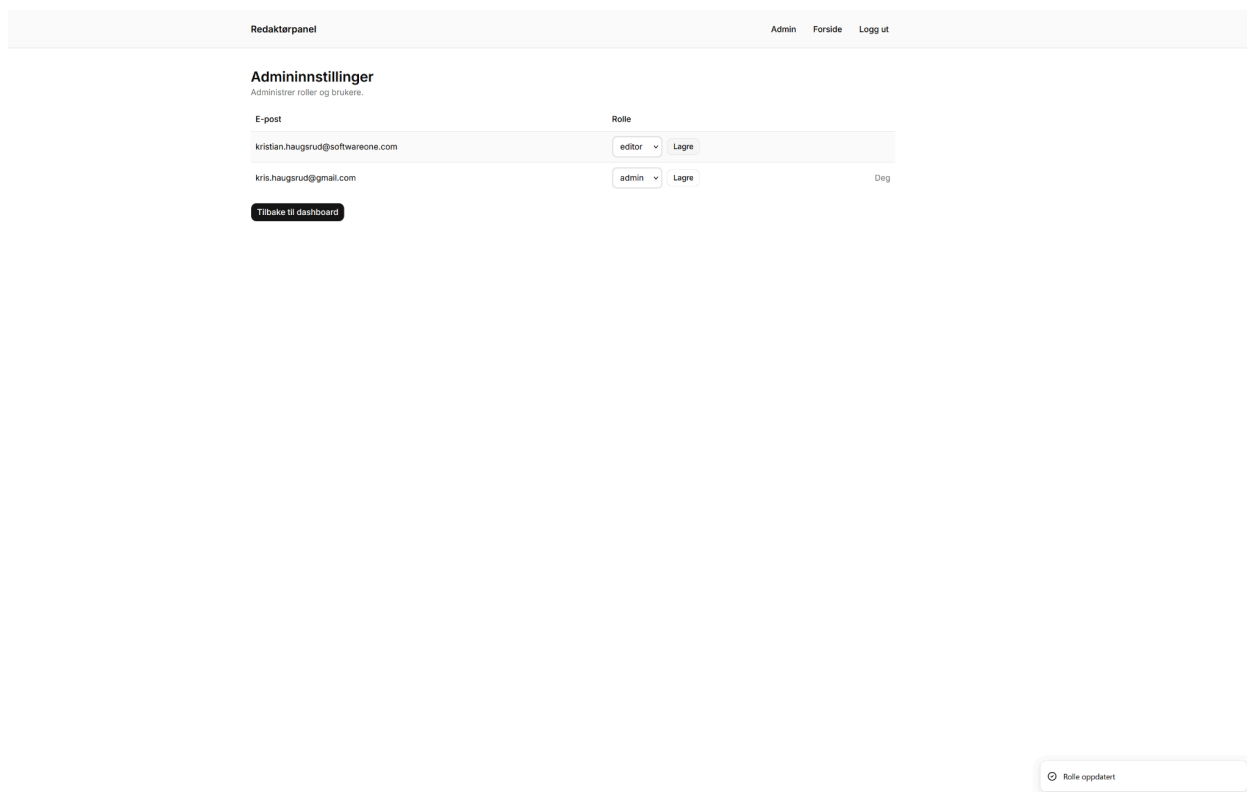
For å komme videre med målene i dag om å få satt opp redaktør sidene velger jeg å legge til brukere manuelt foreløpig via Supabase dashboardet og heller komme tilbake senere. Jeg har også sett på og vurderer å prøve å koble på en ekstern e-postleverandør (for eksempel [Resend](#)) for å ikke møte på problemer om rate limits igjen.

Etter dette begynte jeg å sette opp redaktørpanelet. Jeg satt opp hovedsiden og kortene som skal linke til artikler, nyheter, arrangementer og medlemsregisteret.



Hovedsiden til redaktør dashboardet

I tillegg satt jeg opp en egen admin side for å administrere brukere og roller. Siden er også bare tilgjengelig for brukere som har admin rollen hvis en vanlig redaktør prøver å navigere seg dit blir de redirecta til /admin.



Brukerstyrings panelet for admins

Etter jeg satt opp styring av oversikt over bruker og roller fortsatte jeg med sidene for CRUD operasjoner for nyheter, artikler og arrangementer. Jeg brukte ganske likt oppsett på sidene, jeg satt opp en liste med tabell og knapper for å opprette/redigere/slette, skjema-sider, og server actions som valideres med Zod. det er også satt opp med varslinger og toasts slik at hvis noe feiler får redaktøren beskjed.

Redaktørpanel				Admin	Forside	Logg ut
Arrangementer						
Opprett, rediger og slett arrangementer, og se påmeldte.						
Nytt arrangement Tilbake til dashboard						
Tittel	Dato og klokkeslett		Kategori	Sted	Handling	
fotballtrening	29.05.2026 21:00		fotball	Norge	Rediger	Slett

Arrangementsliste i redaktør panelet

Redaktørpanel				Admin	Forside	Logg ut
Nyheter						
Opprett, rediger og slett nyheter.						
Ny nyhet Tilbake til dashboard						
Tittel	Kategori	Opprettet			Handling	
test	tets	28.5.2026, 15:33:37			Rediger	Slett

Nyhetsliste i redaktør panelet

Redaktørpanel				Admin	Forside	Logg ut
Artikler						
Opprett og rediger infosider som «Om klubben» og «Kontakt».						
Ny infoside Tilbake til dashboard						
Tittel	Sist oppdatert				Handling	
Test artikkel	28.5.2026, 16:04:19				Rediger	Slett

Artikkelliste i redaktør panelet

For arrangementer la jeg også inn visning av påmeldte på rediger-siden, så redaktøren ser hvem som har meldt seg på.

Et problem jeg støttet på når jeg lagde arrangement siden var at dato og tid ikke alltid stemte med norsk tid og brukte engelske tider som AM og PM. Måten jeg fikset det med å bruke enda en Date-fns pakke (date-fns-tz) for å få tiden til å stemme med Europa og Oslo tidssoner. Jeg lagde også en egen TimeSelect komponent med en dropdown som går i 15 minutters intervaller som jeg bruker på arrangement sidene slik at redaktørene lett kan velge tid for arrangementene.

```
lib > TS date.ts > ...
1  import { formatInTimeZone } from "date-fns-tz";
2
3  const OSLO_TZ = "Europe/Oslo";
4
5  export function formatOsloDateTime(iso: string | null | undefined) {
6    if (!iso) return "-";
7    return formatInTimeZone(new Date(iso), OSLO_TZ, "dd.MM.yyyy HH:mm");
8  }
9
10 export function isoToOsloDateAndTime(iso: string | null | undefined) {
11   if (!iso) return { date: "", time: "" };
12   const d = new Date(iso);
13   return {
14     date: formatInTimeZone(d, OSLO_TZ, "yyyy-MM-dd"),
15     time: formatInTimeZone(d, OSLO_TZ, "HH:mm"),
16   };
17 }
18
19 export function generateTimeOptions(
20   startHour = 6,
21   endHour = 22,
22   intervalMinutes = 15,
23 ) {
24   const options: string[] = [];
25   const start = startHour * 60;
26   const end = endHour * 60;
27
28   for (let minutes = start; minutes <= end; minutes += intervalMinutes) {
29     const h = Math.floor(minutes / 60);
30     const m = minutes % 60;
31     options.push(
32       `${String(h).padStart(2, "0")}:${String(m).padStart(2, "0")}`,
33     );
34   }
35
36   return options;
37 }
```

Hjelpfunksjoner får håndtering av norsk dato og tid. ([date-fns-tz dokumentasjon](#)).


```
🔒 USE OPTIONS ABOVE TO EDIT
1  alter policy "news_editor_admin_crud"
2  on "public"."news"
5  to authenticated
6  using (
7    is_editor_or_admin()
8  ) with check (
9    is_editor_or_admin()
10 );
```

✕ Edit 'is_editor_or_admin' function

Name of function

is_editor_or_admin

Name will also be used for the function name in postgres

Schema

schema public

Tables made in the table editor will be in 'public'

Arguments

Arguments can be referenced in the function body using either names or numbers.

No argument for this function

Definition

The language below should be written in sql.

```
1
2  select exists (
3    select 1 from public.profiles p
4    where p.id = auth.uid() and p.role in ('editor', 'admin')
5  );
6
```

Jeg sjekket også om det var feil kobling mellom bruker i Supabase auth og hvordan de ble lagret i Profiles tabellen, men bruker ID-ene matchet så det var ikke problemet.

904ddb5a-76fd-4fa9-b41a-e6f06...	kris.haugsrud@gmail.com	admin
b3501700-9e0f-4d11-8249-82b63...	kristian.haugsrud@softwareone.com	editor
Referencing record from auth.users:		
email varchar	encrypted_password	
kristian.haugsrud@softwareone.com	\$2a\$10\$gBu	

Etter litt mer undersøking og søking i Supabase docs og google fant jeg ut at problemet sannsynligvis hang sammen med hvordan RLS-policyene brukte `is_editor_or_admin()` funksjonen.

Polyciene på news, events, articles og member sjekker om brukeren er editor eller admin med å bruke denne funksjonen. Funksjonen leser fra Profiles tabellen for å finne ut hvilken rolle brukeren har. Jeg fant ut at siden Profiles tabellen har RLS skudd på så prøvde databasen å sjekke rollen flere ganger etter hverandre. Dette skapte en slags loop, og det er derfor jeg fikk "stack depth limit exceeded" i terminalen.

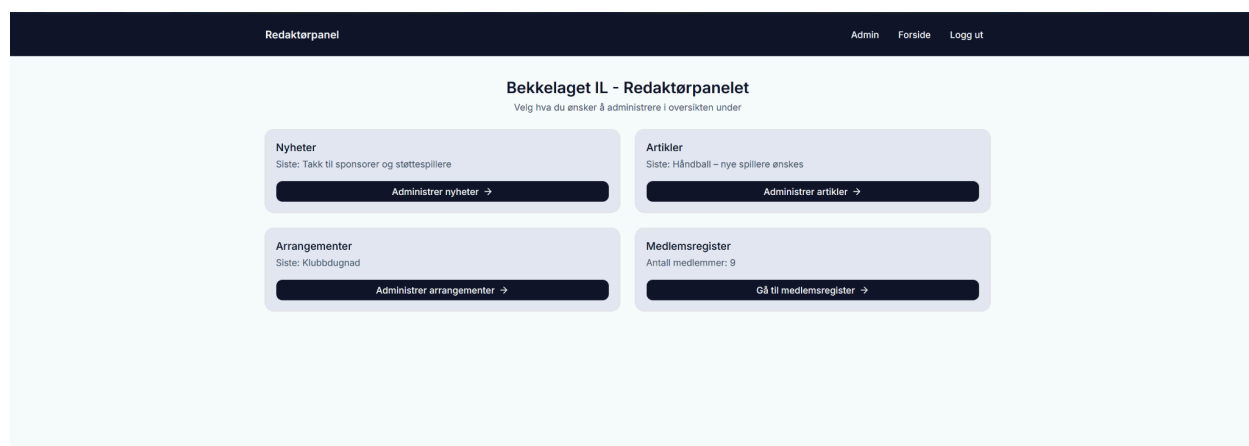
Det forklarer også hvorfor det så ut som at det ikke fantes noe data når man logget inn.

Jeg fant en løsning på det som var å oppdatere `has_role()` og `is_editor_or_admin` funksjonene med SECURITY DEFINER. Sånn jeg har forstått det er SECURITY DEFINER en innstilling i databasen som bestemmer hvem funksjonen kjører som. I mitt tilfelle gjør det at funksjonen kan lese fra profiles tabellen uten at RLS-en starter hele sjekken på nytt. Da slipper jeg problemer med rekursjon og at funksjonen bare looper.

Jeg justerte også policyen på profiles slik at alle innloggede brukere kan lese sin egen profil, mens admin kan lese alle profiler.

Etter jeg gjorde endringene i Supabase logget jeg ut og inn igjen som editor og alt lastet inn som det skulle. Noe jeg kommer til å prøve å ta med etter dette er at jeg må passe på hvordan polyices og funksjoner trigger hverandre når jeg setter de opp slik at jeg ikke havner i en lignende situasjon hvor det tar meg litt tid på å finne feilen.

Etter dette hadde jeg fortsatt litt tid igjen av dagen, så jeg brukte denne på å sette opp farger i prosjektet slik at det er klart til når jeg skal begynne å lage forsiden på mandag.



```

:root {
  --background: #f8fafc;
  --foreground: #0f172a;
  --card: #e2e8f0;
  --card-foreground: #0f172a;
  --popover: #f8fafc;
  --popover-foreground: #0f172a;
  --primary: #0f172a;
  --primary-foreground: #f8fafc;
  --secondary: #e2e8f0;
  --secondary-foreground: #0f172a;
  --muted: #e2e8f0;
  --muted-foreground: #475569;
  --accent: #eab308;
  --accent-foreground: #0f172a;
  --destructive: #ef4444;
  --success: #22c55e;
  --success-foreground: #0f172a;
  --border: #cbd5e1;
  --input: #94a3b8;
  --ring: #0f172a;
  --radius: 0.625rem;
  --sidebar: #f8fafc;
  --sidebar-foreground: #0f172a;
  --sidebar-primary: #0f172a;
  --sidebar-primary-foreground: #f8fafc;
  --sidebar-accent: #eab308;
  --sidebar-accent-foreground: #0f172a;
  --sidebar-border: #e2e8f0;
  --sidebar-ring: #0f172a;
}

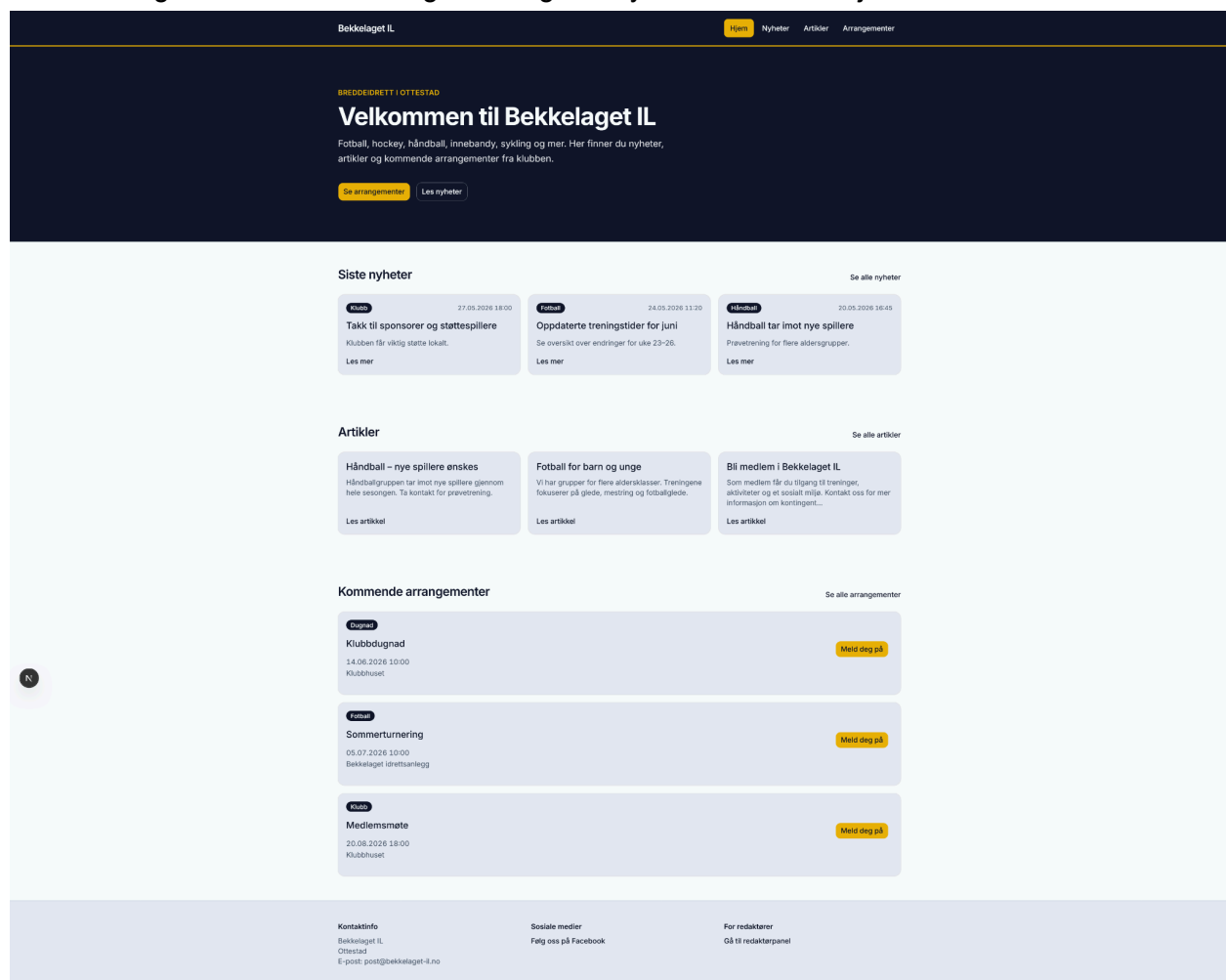
```

Jeg synes dagen har vært veldig produktiv og at jeg ligger veldig godt an ifølge planen. Jeg har også lært en del nytt om hvordan RLS policyer kan funke med hverandre på gode og dårlige måter.

Mandag 1. Juni – Dag 5

Startet dagen med å se gjennom og rydde opp litt i koden fra fredag, det var ikke så mye jeg fant som trengtes å fikses. Så hadde jeg møtet med prøvenemnda, møtet gikk veldig bra, jeg viste fram medlemsregisteret og endringene etter jeg hadde satt opp farger for prosjektet. Jeg fortalt også litt om feilen jeg møtte på med “stack depth limit exceeded” og infinite recursion. Jeg fikk også noen spørsmål om hva som skjedde hvis en redaktør gikk ut av siden mens de skrev en artikkel eller nyhet. Det hadde jeg ikke tenkt på før og var ikke en del av planleggingen. Når jeg tenkte på det burde det nok ha vært funksjonalitet for å ha utkast av de forskjellige innholdstypene og mulighet til å starte å skrive og komme tilbake til det i etterkant.

Etter møtet begynte jeg å lage forsiden til siden. Jeg prøvde å sette opp layouten slik jeg hadde tenkt med figma skissene mine og er veldig fornøyd med hvordan hjemmesiden ble.



Etter jeg var ferdig med hovedsiden gikk jeg over til å lage lisensidene for artikler og nyheter. De fungerer veldig likt og lister ut alt innhold, man kan søke og på nyheter har man mulighet til å

filtrere på kategori.

Bekkelaget IL

HjemNyheterArtiklerArrangementer

Nyheter

Nyheter og oppdateringer fra Bekkelaget IL.

Alle kategorier ▾

Klubb27.05.2026 18:00

Takk til sponsorer og støttespillere

Klubben får viktig støtte lokalt.

Les mer

Fotball24.05.2026 11:20

Oppdaterte treningstider for juni

Se oversikt over endringer for uke 23–26.

Les mer

Håndball20.05.2026 16:45

Håndball tar imot nye spillere

Prøvetrening for flere aldersgrupper.

Les mer

Dugnad18.05.2026 08:15

Dugnad på klubbhuset – vi trenger deg

Lørdag samles vi for vårrengjøring og klargjøring.

Les mer

Fotball14.05.2026 14:30

Sterk helg for fotballgruppen

Flere lag leverte gode prestasjoner.

Les mer

Klubb10.05.2026 09:00

Velkommen til ny sesong i Bekkelaget IL

Klubben er i gang med aktiviteter for vår og sommer.

Les mer

Bekkelaget IL

HjemNyheterArtiklerArrangementer

Artikler

Artikler og informasjon fra Bekkelaget IL.

Håndball – nye spillere ønskes

Håndballgruppen tar imot nye spillere gjennom hele sesongen. Ta kontakt for prøv...

Les artikkel

Fotball for barn og unge

Vi har grupper for flere aldersklasser. Treningene fokuserer på glede, mestring...

Les artikkel

Bli medlem i Bekkelaget IL

Som medlem får du tilgang til treninger, aktiviteter og et sosialt miljø. Kontak...

Les artikkel

Mens jeg satt opp sidene pratet jeg litt med en kollega som lurte på hvor langt jeg hadde kommet. Jeg viste fram hva jeg hadde gjort så langt, han spurte om jeg hadde satt opp noe caching på dataene som ble hentet fra supabase. Jeg har egentlig ikke tenkt på caching i det hele tatt. Han sa jeg burde se på LRU Cache (Least Recently Used) og at det kunne passe prosjektet. Jeg tok meg litt tid til å se på det og sånn jeg forstår det så funker en LRU cache med at den lagrer de mest populære og nylig forespurte dataene i minnet, og automatisk kaster ut de dataene som er minst brukt når cachen blir full. Det ville gjort at vi ikke måtte ha sendt en

Fagprøve | IT-Utvikling

26.5 - 3-6.2026

39

forespørsel til Supabase hver gang man laster inn en side slik at dataene blir lagret. Etter å ha sett litt på dokumentasjon om LRU cache og sett hvordan jeg ligger an tror jeg at det ikke vil gå å legge det til nå. Jeg har egentlig lite erfaring når det kommer til å sette opp og jobbe med caching, så det blir nok en del å sette seg inn i nå midt i fagprøva. Og med tanke på at jeg har 1-2 dager igjen og jeg skal lage ferdig det jeg allerede har planlagt så tror jeg det vil ta litt for lang tid. Jeg velger derfor å prioritere å lage ferdig det jeg har planlagt, men kommer til å ta det med meg videre og legge det til som et punkt for videre utvikling.

Etter det satt jeg opp listesiden for arrangementer og de individuelle sidene der man kan melde seg på. Og med sjekk en sjekk slik at det ikke blir lagt til to personer med samme mail på samme arrangement.

← Tilbake til arrangementer

Klubb

20.08.2026 18:00

Medlemsmøte

Klubbhuset

Informasjonsmøte for medlemmer og foreldre.

Meld deg på

Fyll inn navn og e-post. Du trenger ikke brukerkonto.

Du er påmeldt. Vi gleder oss til å se deg!

Meld deg på

Fyll inn navn og e-post. Du trenger ikke brukerkonto.

Navn

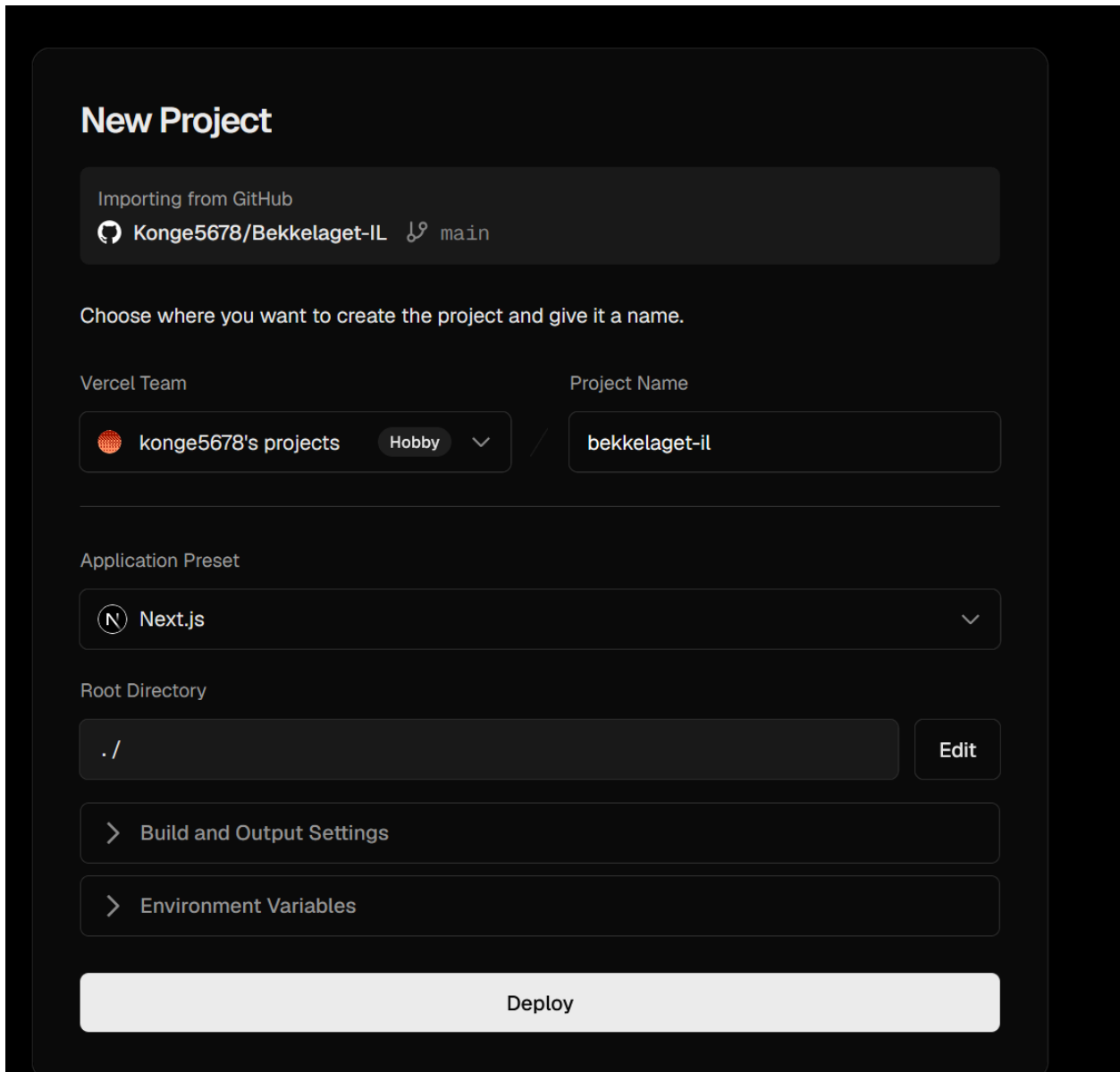
E-post

Telefon (valgfritt)

Send påmelding

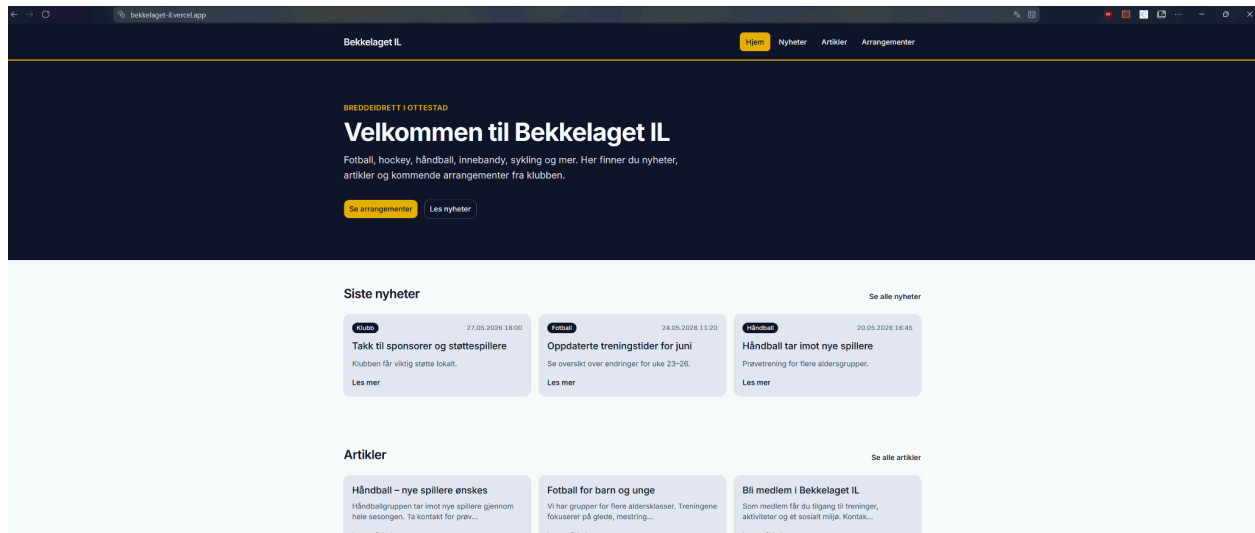
Du er allerede påmeldt dette arrangementet med denne e-postadressen.

Etter jeg hadde satt opp arrangementer var det meste på forsiden satt opp, så jeg brukte en del tid på testing av at alt fungerte som det skulle og at fargepaletten ble brukt hele veien. Mot slutten av dagen skulle jeg deploye prosjektet til Vercel.



The screenshot shows the 'New Project' interface on Vercel. At the top, it says 'New Project'. Below that, a box indicates 'Importing from GitHub' with the repository 'Konge5678/Bekkelaget-IL' and the 'main' branch. A prompt asks to 'Choose where you want to create the project and give it a name.' The 'Vercel Team' section shows 'konge5678's projects' with a 'Hobby' plan selected. The 'Project Name' is 'bekkelaget-il'. The 'Application Preset' is 'Next.js'. The 'Root Directory' is './'. There are expandable sections for 'Build and Output Settings' and 'Environment Variables'. A large 'Deploy' button is at the bottom.

Det var en ganske enkel prosess. Jeg logget inn på vercel, valgte create new, valgte riktig github repo, la inn env variabler og så var det oppe. Det bygde uten noen feil, og siden fungerer fint på <https://bekkelaget-il.vercel.app/>



Jeg synes at dagen i dag gikk veldig bra, jeg er fornøyd med hvor langt jeg har kommet. Ligger godt an ifølge planen og imorgen kommer jeg til å teste litt skikkelig på all funksjonalitet på sidene og prøve å få noen på kontoret til å teste for meg også slik at jeg passer på at ting funker, og kanskje få litt feedback på design ui/ux.

Tirsdag 2. Juni – Dag 6

Dagen startet med siste status med prøvenemnda som jeg synes gikk veldig bra. Pratet om gårdsdagen og hvor jeg lå an. Jeg ligger veldig godt an ifølge planen jeg satt opp føler at ting begynner å bli klart.

Jeg har igjen litt testing og prøve å få noen andre på kontoret til å teste det for meg.

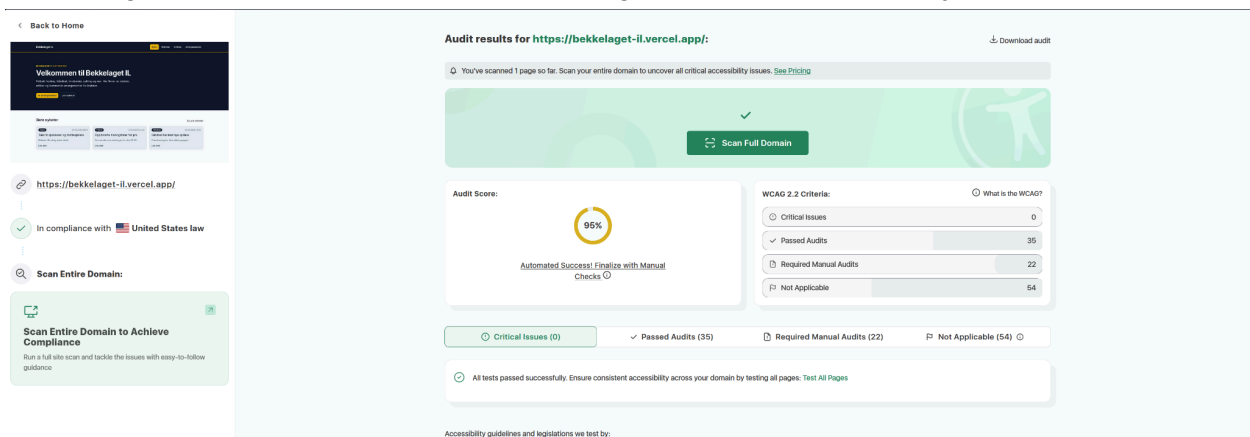
Etter møtet pratet jeg med noen kollegaer for å teste og få litt feedback på hva de synes. Det var ingen funksjonalitet som ikke funkete så det var veldig bra, men en felles tilbakemelding fra to kollegaer handlet om cookies, hvordan jeg lagrer dem og om at vi burde varsle om cookies på siden for å følge GDPR lover. Vi håndterer ikke cookies på forsiden for vanlige brukere og i redaktør panelet lagrer vi bare session cookies for å holde redaktører og admins logget inn. Vi har ikke analyse eller springer og vi kjører ingen markedsføring så det er ingen spesiell grunn for cookie godkjenning. Men en ting siden absolutt burde hatt var en egen side for personvernerklæring. En personvernerklæring er lovpålagt hvis siden behandler personopplysninger og med pålogging til arrangementer gjør siden min det. Så hvis jeg hadde hatt bedre tid hadde jeg satt opp en egen side som beskriver hvilke opplysninger som samles inn, formålet med det vi lagrer, hvordan de blir lagret på en sikker måte, hvor lenge vi lagrer dem og hvordan de kan få dataene slettet hvis de ønsker det.

En annen ting jeg fikk feedback på er at per nå så er det ingen måte for brukere som har meldt seg på et arrangement for å få informasjonen sin slettet hvis de ønsker. Dette er også noe jeg burde ha tenkt på med tanke på GDPR og håndtering av personlig informasjon og er noe man

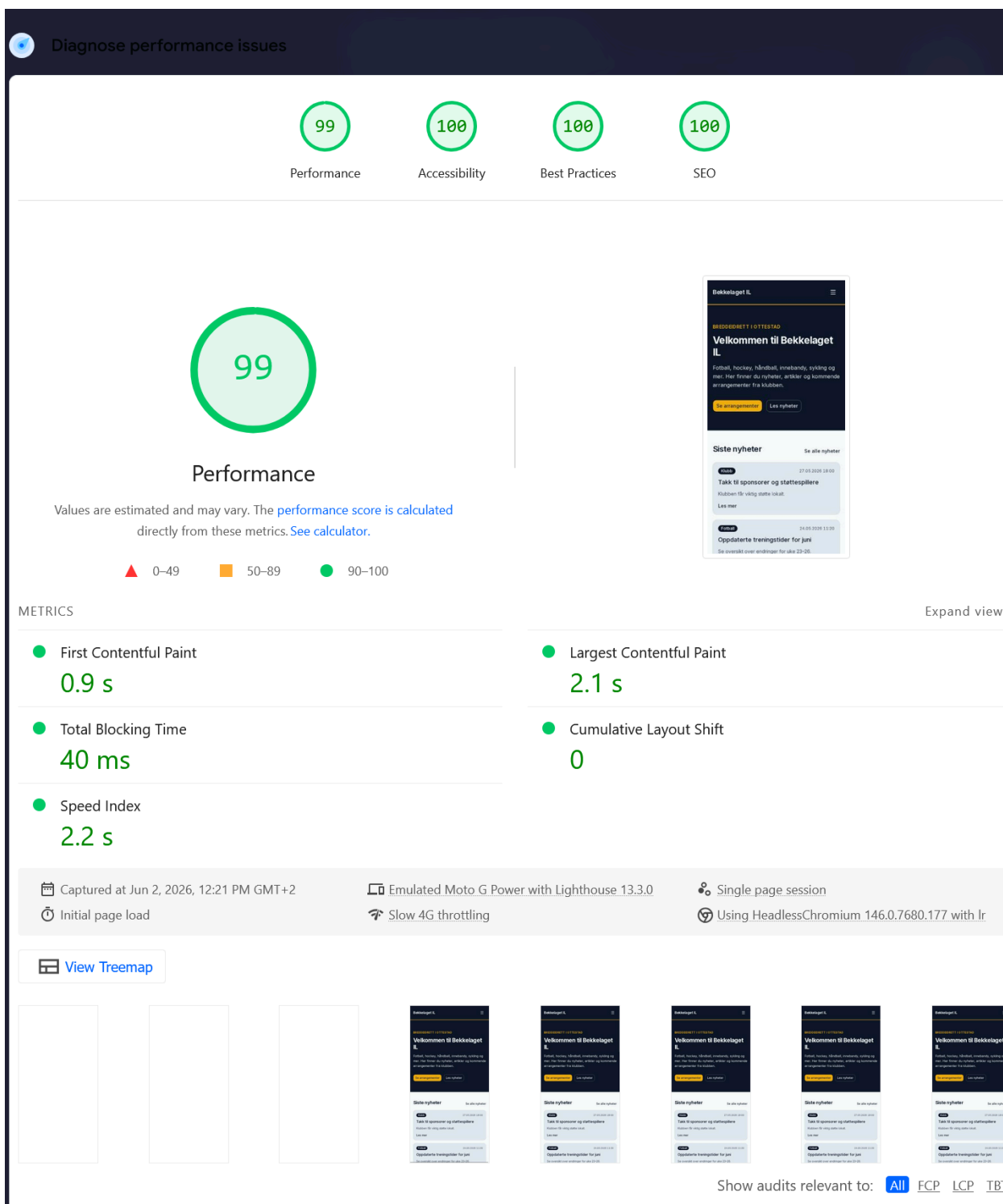
kunne ha skrevet inn i en personvernerklæring om hvordan man kontakter for å få personlig informasjon slettet.

Jeg føler at dette er en litt større ting som jeg burde ha tatt med inn i planleggingen min, men kommer til å ta det meg meg videre om å passe mer på lovverkene rundt datahåndtering.

Etter det brukte jeg litt tid på å teste siden selv, jeg kjørte nettsiden gjennom tester på WCAG for å teste at løsningen tilfredsstiller krav om universell utforming. Siden skal bli brukt av mange forskjellige mennesker så det er viktig at siden er oversiktlig, raskt og tilgjengelig. Jeg kjørte tester på [Google Lighthouse](https://lighthouse.google.com/) og accessibilitychecker.org. Testene kom tilbake veldig positive. Jeg treffer ekstremt høyt på lighthouse som viser at siden laster inn raskt og at siden vil være lett å laste inn for brukere med tregere nettverksforhold. Jeg fikk også en score på 95% på accessibility checker. Det viser at nettsiden er bygd opp på en logisk måte, og at den treffer bra på ting som alt-tekst på bilder, ledetekster og universelt utformede skjemaer.



accessibilitychecker.org sjekk



Resten av dagen ble brukt hovedsakelig på å skrive ferdig dokumentasjon og å skrive egenvurdering. Jeg begynte også å planlegge hvordan jeg skulle vise fram til prøvenemnda imorgen.

Onsdag 3. Juni – Dag 7

Denne dagen jobbet jeg med å fint pusse dokumentasjonen og sette opp et dokument for å hjelpe meg med framføring.

Egenvurdering

Jeg er stolt over alt jeg har oppnådd under denne prosessen og tenker selv at oppgaven gikk veldig bra. Det har vært en stor læringsprosess der jeg har fått testet meg på teknologier og verktøy jeg i utgangspunktet ikke var så trygg på, noe som har gitt meg en dypere forståelse for faget. Hvis jeg skal trekke frem det jeg er aller mest fornøyd med, så er det måten jeg har satt opp databasen og Row Level Security) i prosjektet på. Jeg har ikke så mye erfaring med backend fra før og det eneste ordentlige prosjektet jeg hadde satt opp en backend struktur for tidligere var prøvefagprøven min, så jeg føler at jeg har utviklet meg og lært mye og backend konsepter som jeg ikke var så godt kjent med tidligere. Jeg er også veldig fornøyd med oppsettet av farger og responsiviteten på siden, jeg prøvde å tenke mye på det i planleggingen med å sette opp fargepalett og lage skisser, og føler det hjalp meg med å lage et veldig fint og intuitivt design for brukerne.

Det jeg er minst fornøyd med er at å legge til nye redaktører ikke er like intuitivt som jeg ønsket i planleggingen. Jeg møtte på feil med rate limits og per nå må man inn i Supabase dashboardet for å legge til nye redaktører. Det fungerer for en MVP som dette, men i en helhetlig løsning burde en slik flyt ha blitt satt opp bedre.

Hva jeg har lært

Jeg føler hele prosjektet har vært en god læringsprosess for meg. Jeg har gått dypere inn i verktøy og teknologi jeg har testet tidligere, men som jeg nå føler jeg har en mye bedre forståelse for.

Det området jeg har fått fordypet meg aller mest i er backend. Jeg bruker Supabase for backend og database. I starten av prosjektet hadde jeg bare brukt Supabase til enkle løsninger før og satt opp autentisering, men jeg har aldri gått så dypt i hva man må gjøre for å holde en tjeneste sikker. Jeg har lært mye om konsepter som RLS, CORs og policys. For eksempel fikk jeg en skikkelig utfordring på fredagen med en RLS-loop som ga feilmeldingen "stack depth limit exceeded", og jeg lærte utrolig mye av å feilsøke meg frem til at løsningen var å bruke SECURITY DEFINER på funksjonene i databasen.

Jeg har også fordypet meg i hvordan autentisering fungerer i Next.js. Spesielt har jeg blitt bedre på hvordan man bruker middleware for å beskytte ruter. Og jeg føler at jeg har utviklet evnen min til å lage WCAG vennlige sider.

Hva jeg kunne gjort annerledes

Hvis jeg skulle startet på prosjektet helt på nytt med det jeg vet nå, ville jeg nok brukt litt mer tid i starten på å se på hvordan innholdet skulle styres. Som jeg diskuterte med prøvenemnda under statusmøtet på torsdag, kunne det vært en bedre løsning å bruke et CMS som Sanity for nyheter og artikler, og heller koble det sammen med medlemsregisteret i Supabase. Da hadde det kanskje blitt enda lettere for redaktørene å jobbe.

Jeg burde nok også ha sett litt nærmere på hva man får i gratisversjonen fra supabase slik at problemet med rate limits i supabase hadde vært klarere. En annen ting jeg ser jeg burde hatt med i planen fra start, var en funksjon for utkast (drafts) så redaktørene ikke mister tekst hvis de lukker vinduet, og å ha planlagt en side for personvernerklæring i planleggingen.

Planleggingen

Jeg føler at planleggingen i forkant gikk veldig bra, og dagsplanen jeg satte opp har vært til stor hjelp for å holde fremdriften oppe gjennom hele uka. Å bruke Jira med kanban boards funket veldig bra til å holde oversikt over hva jeg måtte gjøre hver dag. Jeg synes også at valgene mine om å bruke Next.js, TypeScript og Supabase passet bra for denne oppgaven.

Gjennomføringen

Selve gjennomføringen lider litt av samme problemer som planleggingen med at jeg burde ha undersøkt ting litt dypere. Rate limits i supabase burde jeg ha undersøkt fra før, jeg burde nok også ha satt meg litt mer inn i hvordan RLS Polycier reagerer med hverandre. Hvis jeg hadde tenkt meg litt mer nøye gjennom oppsettet av polycies kunne jeg ha unngått RLS-loopen som dukket opp på fredag.

Annet enn det synes jeg gjennomføringen gikk veldig bra. Jeg er veldig fornøyd med hvordan forsiden ser ut og funker og føler jeg fikk vist mye av designvennene mine. Deployment til Vercel gikk helt uten problemer, og jeg føler at jeg har klart å lage en god MVP.

Plan for videre utvikling

Hvis jeg hadde hatt noen dager til på meg, ville dette vært det første jeg tok tak i:

GDPR og personvern

Sette opp en egen side for personvernerklæring og lage en funksjonalitet så brukere kan slette sine egne opplysninger automatisk, uten å måtte kontakte en administrator.

E-posthåndtering

Koble løsningen opp mot en ekstern e-posttjeneste som Resend. I dag bruker systemet de innebygde funksjonene til Supabase, men for en produksjonsklar side vil en ekstern tjeneste fjerne unødvendige begrensninger (rate-limits).

Eksport av medlemsregisteret

Gjøre det mulig for administratorer å eksportere medlemslistene til et universelt format som CSV. Dette vil gjøre det langt enklere for ledelsen å hente ut rådata, bearbeide informasjonen i eksterne programmer som Excel, eller bruke listene til andre administrative formål.

Maks antall plasser og venteliste

Implementere en funksjon for å sette en grense på antall deltakere per arrangement. Hvis et arrangement blir fullt, bør systemet automatisk kunne håndtere en venteliste og flytte folk opp i køen dersom noen melder seg av. Dette vil gjøre at vi ikke får for mange meldt på et arrangement.

Bildeopplasting og mediebehandling

Sette opp funksjonalitet for å laste opp og lagre filer direkte i databasen. Det vil la redaktører og administratorer laste opp egne bilder til artikler og arrangementer, noe som gjør innholdet på nettsiden mer levende.

Endringslogg for redaktører

Utvikle en egen oversikt i redaktørgrensesnittet som logger nøyaktig hvem som har gjort endringer, hva som ble endret, og når det skjedde. Det gir bedre internkontroll, hindrer misforståelser og gjør det lett å spore opp feil hvis noe skulle gå galt.

Admin-dashboard med statistikk

Bygge en visuell startside for ledelsen og administratorer, fylt med enkle grafer og statistikk over påmeldinger og medlemstrender. Dette vil gi en rask og oversiktlig status på hvordan tjenesten blir brukt, uten at man må lete manuelt gjennom tabellene i databasen.

Læremål

Under er en liste med begrunnelse med alle læremål jeg føler jeg har oppnådd i løpet av oppgaveprosessen

Planlegge, utvikle og dokumentere løsninger med innebygd personvern og sikkerhet

Tjenesten sikrer sensitive personopplysninger i medlemsregisteret ved å bruke et tolags sikkerhetssystem. I frontenden brukes Next.js Middleware til å sperre uautorisert tilgang til alle ruter under /admin Hvis noen prøver å manipulere forespørsler i nettleseren, stopper databasen

dette nede på tabellnivå med Row Level Security policies i Supabase. I tillegg er prinsippet om dataminimering fulgt, slik at det kun samles inn helt nødvendig kontaktinformasjon

Utvikle og bruke dokumentasjon og veiledninger

Tjenesten har en README-fil som dokumenterer valg av teknologi og oppsett i både frontend og backend. Jeg har også dokumentert arkitekturvalg og hvordan man kloner og kjører prosjektet, slik at onboarding av nye utviklere vil gå fint. Under utviklingen har jeg tatt i bruk flere dokumentasjoner som blant annet:

- <https://nextjs.org/docs>
- <https://supabase.com/docs>
- <https://zod.dev/>

Dette er bare noen av dokumentasjonene jeg brukte under utviklingen.

Velge og bruke relevante rammeverk og moduler til utvikling

Jeg har valgt Next.js som rammeverk og Supabase som backend fordi disse verktøyene tillater rask utvikling uten at det går på bekostning av sikkerhet eller ytelse. Ved å bruke moduler som shadcn/ui har jeg kunnet bygge et profesjonelt grensesnitt effektivt, mens Zod har blitt brukt til skjemavalidering for å sikre at dataene som sendes til databasen alltid er i riktig format

Feilsøke kode og rette feil i algoritmer og kode

Jeg føler at feilsøking av kode og å rette feil er noe som skjer naturlig når man utvikler en tjeneste. Det er ikke så mye forklaring jeg føler er nødvendig å skrive ned hvorfor dette punktet er oppfylt.

Utvikle og tilpasse brukergrensesnitt som ivaretar krav til universell utforming

For å sikre at tjenesten er universelt utformet uten at det stjaler for mye tid fra utviklingen, valgte jeg å bruke komponentbiblioteket shadcn/ui. Siden disse komponentene er bygget på Radix UI, får jeg kritisk funksjonalitet som tastaturnavigasjon og støtte for skjermlesere helt gratis. Dette gjør at jeg kan levere en profesjonell løsning som følger kravene på en effektiv måte, i stedet for å bruke tid på å kode tilgjengelighet fra bunnen av. Jeg har i tillegg passet på at fargekontraster og responsivt design fungerer på alle skjermstørrelser.

Håndtere påloggingsopplysninger på en sikker og forsvarlig måte

Påloggingen håndteres gjennom Supabase. All påloggingslogikk, kryptering og registrering skjer gjennom deres hjelpfunksjoner. Dette er en trygg løsning som fjerner risikoen for at jeg ved et uhell misbehandler brukerdata. Passord blir kryptert og er aldri tilgjengelig for meg eller andre brukere. Jeg har også passet å ikke lagre mer data enn nødvendig, bortsett fra e-post og passord lagres kun brukerens rolle